Séance 3 — Rendre l'application intelligente et robuste

Sommaire

Comment exécuter les exemples

1. Des interfaces aux lambdas
   - 1.1 🟢 Point de départ : `Discount`, une interface à une seule méthode
   - 1.2 🟢 Pourquoi ? Une classe entière pour une ligne de logique
   - 1.3 🔵 Étape intermédiaire : la classe anonyme
   - 1.4 🟢 Comment ? La lambda
   - 1.5 🟢 Syntaxe et capture de variables
   - 1.6 🟢 Les interfaces fonctionnelles du JDK
   - 1.7 🟢 Les références de méthodes (et enfin `Integer::sum`)
2. L'API Stream
   - 2.1 🟢 Pourquoi ? Dire *quoi*, pas *comment*
   - 2.2 🟢 Anatomie d'un pipeline
   - 2.3 🟢 `filter`, `map`, `toList`
   - 2.4 🟢 `sorted` et `limit` : le top 3 des commandes
   - 2.5 🟢 `count`, `anyMatch`, `allMatch`, `findFirst`
   - 2.6 🟢 Chiffre d'affaires : `reduce` avec `BigDecimal`
   - 2.7 🟢 Grouper : `Collectors.groupingBy` et `counting`
   - 2.8 🟢 Les trois pièges des streams
   - 2.9 🔵 `flatMap` : les plats les plus commandés
   - 2.10 🔵 `Collectors.toMap` et les collisions de clés
   - 2.11 🔵 Streams parallèles : pourquoi les éviter par défaut
   - 2.12 🟣 Gatherers (JDK 24) : des opérations intermédiaires sur mesure
3. Les exceptions
   - 3.1 🟢 Pourquoi ? Une erreur doit se voir
   - 3.2 🟢 La hiérarchie : checked ou unchecked
   - 3.3 🟢 `try` / `catch` / `finally` et le multi-catch
   - 3.4 🟢 try-with-resources
   - 3.5 🟢 Une exception métier : `OrderNotFoundException`
   - 3.6 🟢 `getOrderById` ou `findOrderById` : lever ou renvoyer un `Optional` ?
   - 3.7 🟢 Valider les transitions : `IllegalStateException`
   - 3.8 🟢 Ne jamais avaler une exception, ni renvoyer `0` ou `null` « par défaut »
   - 3.9 🔵 Envelopper une exception technique sans perdre la cause
   - 3.10 🟣 Exceptions checked dans les lambdas
   - 3.11 🟣 La variable anonyme `_` (Java 22)

4. Journaliser sans fuir
   4.1 🟢 `System.Logger` plutôt que `printStackTrace`
   4.2 🔒 Fuite d'information par les messages d'erreur (CWE-209)
   4.3 🔒 Pas de données personnelles dans les logs
   4.4 🔒 Injection dans les logs (CWE-117)
5. Lire et écrire des fichiers
   5.1 🟢 `Path` : désigner un fichier
   5.2 🟢 `Files` : lire et écrire du texte
   5.3 🟢 Les flux d'octets : `newOutputStream` et `newInputStream`
   5.4 🔵 Écrire de façon atomique
   5.5 🔒 Un nom de fichier choisi par l'utilisateur (CWE-22)
   5.6 🟢 Tester avec `@TempDir`
6. Ce qu'il faut retenir
7. Pour aller plus loin

Annexe A — Le modèle FoodFast de cette séance
   A.1 `DishSize`
   A.2 `OrderStatus`
   A.3 `Dish`
   A.4 `User`
   A.5 `Customer`
   A.6 `Order`
   A.7 `OrderNotFoundException`
   A.8 `DeliveryPlatform`
   A.9 `SampleData`

> **Objectifs** :
>
> - **AA5** : transformer et agréger des données avec les lambdas et l'API Stream.
> - **AA6** : gérer les erreurs avec des exceptions métier, sans masquer les erreurs ni divulguer d'information.
>
> **Prérequis** : séance 1 (bases du langage) et séance 2 (classes, encapsulation, `record`, `enum`, `BigDecimal`, héritage, interfaces et `Discount`, collections, `Optional`).
>
> **Comment lire ce support** : 🟢 indispensable · 🔵 pour approfondir · 🟣 bonus · 🔒 sécurité

En séance 2, vous avez construit le **modèle** de FoodFast : des plats, des clients, des commandes, une plateforme qui les range dans une `Map`. Ce modèle est solide, mais il est encore « muet » : il ne sait pas répondre à des questions comme « quel est le chiffre d'affaires du jour ? » ou « combien de commandes sont en attente ? ». Et il ne dit rien de ce qui se passe quand quelque chose tourne mal.

Cette séance s'attaque aux deux sujets :

1. **Des interfaces aux lambdas** : passer du comportement « emballé dans une classe » au comportement passé en paramètre.
2. **L'API Stream** : interroger les collections de façon déclarative.
3. **Les exceptions** : signaler une erreur proprement, et ne jamais la masquer.
4. **Journaliser sans fuir** : `System.Logger`, et les erreurs de sécurité classiques autour des messages d'erreur et des logs.

Comment exécuter les exemples

Chaque exemple complet commence par `// Fichier : Nom.java`. Il a été compilé et exécuté avec le JDK 25.

- Un exemple **autonome** se lance directement : `java Nom.java`.
- Un exemple marqué **« avec … »** s'appuie sur les fichiers du modèle, en **annexe A** (fin du document). Placez ces fichiers dans le même dossier que l'exemple, puis lancez `java Nom.java` : depuis Java 22, le lanceur compile automatiquement les autres fichiers source du dossier (JEP 458).

Les données de démonstration (`SampleData`, annexe A.9) contiennent six commandes :

Client	Contenu	Total	Statut
Alice	2 Margherita, 1 Tiramisu	30.50	`DELIVERED`
Bob	1 Burger, 1 Tiramisu	17.40	`DELIVERED`
Alice	1 Salade César	9.80	`PENDING`
Bob	3 Margherita	37.50	`IN_DELIVERY`
Alice	1 Burger, 2 Tiramisu	22.90	`CANCELLED`
Bob	2 Burger	23.80	`DELIVERED`

---

1. Des interfaces aux lambdas

1.1 🟢 Point de départ : `Discount`, une interface à une seule méthode

En séance 2, vous avez écrit l'interface `Discount` et deux implémentations :

```
interface Discount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount { /* ... */ }
record FixedDiscount(BigDecimal amount) implements Discount { /* ... */ }
```

Regardez bien `Discount` : elle ne déclare **qu'une seule méthode abstraite**. Tout ce qu'elle décrit, c'est « une façon de transformer un total en un autre total ». Autrement dit, `Discount` ne représente pas vraiment un *objet* (avec un état, une identité) : elle représente un **comportement**.

C'est exactement le type d'interface que Java sait raccourcir avec une **lambda**. Avant d'y arriver, voyons le problème.

1.2 🟢 Pourquoi ? Une classe entière pour une ligne de logique

Le problème. Le service marketing invente une nouvelle promotion chaque semaine : « happy hour, 2 € de remise », « livraison offerte », « -15 % le lundi »… Avec ce que vous savez faire, chaque règle demande une classe complète :

```java
// Fichier : DiscountClasses.java
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.List;

interface Discount {
    BigDecimal apply(BigDecimal total);
}

record PercentageDiscount(int percent) implements Discount {
    public BigDecimal apply(BigDecimal total) {
        return total.multiply(BigDecimal.valueOf(100 - percent))
                    .divide(BigDecimal.valueOf(100), 2, RoundingMode.HALF_UP);
    }
}

/** Une classe nommée... pour une seule ligne de logique. */
class HappyHourDiscount implements Discount {
    @Override
    public BigDecimal apply(BigDecimal total) {
        return total.subtract(new BigDecimal("2.00")).max(BigDecimal.ZERO);
    }
}

void main() {
    BigDecimal total = new BigDecimal("30.50");
    List<Discount> discounts = List.of(new PercentageDiscount(10), new
HappyHourDiscount());
    for (Discount d : discounts) {
        IO.println(d.getClass().getSimpleName() + " -> " + d.apply(total));
    }
    // PercentageDiscount -> 27.45
    // HappyHourDiscount -> 28.50
}
```

Pour une ligne utile (`total.subtract(...)`), il a fallu : un nom de classe, un `implements`, un `@Override`, une signature complète. Multipliez par vingt promotions : le code métier disparaît sous la « cérémonie ».

1.3 🔵 Étape intermédiaire : la classe anonyme

Avant Java 8, on pouvait éviter de **nommer** la classe grâce à une **classe anonyme** : on déclare et on instancie la classe en une seule expression, à l'endroit où on en a besoin.

```java
// Fichier : DiscountAnonymous.java
import java.math.BigDecimal;

interface Discount {
    BigDecimal apply(BigDecimal total);
}

void main() {
    // Classe anonyme : "une classe sans nom qui implémente Discount"
    Discount happyHour = new Discount() {
        @Override
        public BigDecimal apply(BigDecimal total) {
            return total.subtract(new BigDecimal("2.00")).max(BigDecimal.ZERO);
        }
    };
    IO.println(happyHour.apply(new BigDecimal("30.50"))); // 28.50
}
```

On a gagné un fichier, mais pas beaucoup de lisibilité : cinq lignes de structure pour une ligne de logique. Vous croiserez encore des classes anonymes dans du code ancien (interfaces graphiques Swing, vieux tests). Elles restent utiles dans un cas : quand l'interface a **plusieurs** méthodes abstraites, une lambda est impossible.

1.4 🟢 Comment ? La lambda

Une **lambda** est une fonction anonyme : on n'écrit plus que **les paramètres et le corps**. Le compilateur déduit le reste (le type `Discount`, le nom de la méthode `apply`, les types des paramètres) à partir du contexte.

```java
// Fichier : DiscountLambda.java
import java.math.BigDecimal;
import java.util.List;

interface Discount {
    BigDecimal apply(BigDecimal total);
}

void main() {
    BigDecimal total = new BigDecimal("30.50");

    // La même règle que HappyHourDiscount, en une ligne
    Discount happyHour = t -> t.subtract(new
BigDecimal("2.00")).max(BigDecimal.ZERO);

    // Livraison offerte : 2.99 de remise, plafonnée au total
    Discount freeDelivery = t -> t.subtract(new
BigDecimal("2.99")).max(BigDecimal.ZERO);

    // Aucune remise : la fonction identité
    Discount none = t -> t;

    List<Discount> discounts = List.of(happyHour, freeDelivery, none);
    for (Discount d : discounts) {
        IO.println(d.apply(total));
    }
    // 28.50
    // 27.51
    // 30.50
}
```

Ce qu'il faut comprendre :

- `t -> t.subtract(...)` **est** un objet de type `Discount`. On peut le ranger dans une variable, dans une `List<Discount>`, le passer en paramètre : c'est du polymorphisme, exactement comme en séance 2.
- Le compilateur sait que `t` est un `BigDecimal` parce que la méthode `apply` de `Discount` prend un `BigDecimal`. C'est l'**inférence de type**.
- Une lambda ne peut implémenter qu'une interface à **une seule méthode abstraite** : sinon, le compilateur ne saurait pas quelle méthode elle remplace.

Une telle interface s'appelle une **interface fonctionnelle**. On peut (ce n'est pas obligatoire) l'annoter `@FunctionalInterface` : le compilateur refuse alors qu'on lui ajoute une deuxième méthode abstraite par erreur.

```java
@FunctionalInterface
interface Discount {
    BigDecimal apply(BigDecimal total);
}
```

> **Quand garder une classe ou un `record` ?** Quand la règle a des **paramètres** qui font partie de son identité (`PercentageDiscount(10)` et `PercentageDiscount(20)` sont deux remises différentes, comparables avec `equals`), ou quand elle mérite un nom et des tests dédiés. La lambda est faite pour les comportements **courts et locaux**.

1.5 🟢 Syntaxe et capture de variables

Les formes d'une lambda :

```java
// Fichier : LambdaSyntax.java
import java.math.BigDecimal;
import java.util.Comparator;
import java.util.function.Supplier;

void main() {
    // Aucun paramètre : parenthèses vides
    Supplier<String> newId = () -> "order-" + java.util.UUID.randomUUID();

    // Un paramètre : parenthèses facultatives
    java.util.function.Function<BigDecimal, BigDecimal> withTip = t ->
t.add(new BigDecimal("1.00"));

    // Deux paramètres : parenthèses obligatoires
    Comparator<BigDecimal> byAmount = (a, b) -> a.compareTo(b);

    // Corps sur plusieurs lignes : accolades et return explicite
    java.util.function.Function<BigDecimal, String> label = t -> {
        if (t.compareTo(new BigDecimal("50")) >= 0) {
            return "grosse commande";
        }
        return "commande standard";
    };

    IO.println(newId.get().startsWith("order-"));              // true
    IO.println(withTip.apply(new BigDecimal("9.80")));          // 10.80
    IO.println(byAmount.compare(BigDecimal.ONE, BigDecimal.TEN)); // -1
    IO.println(label.apply(new BigDecimal("62.00")));           // grosse
commande
}
```

Piège : une lambda ne peut pas modifier une variable locale. Une lambda peut *lire* les variables locales qui l'entourent (on dit qu'elle les **capture**), à condition qu'elles soient **effectivement finales** : jamais réaffectées après leur initialisation.

```
// ❌ Ne compile pas
int pendingCount = 0;
orders.forEach(o -> {
    if (o.getStatus() == OrderStatus.PENDING) {
        pendingCount++; // error: local variables referenced from a lambda
expression
                        //        must be final or effectively final
    }
});
```

Pourquoi cette règle ? Une lambda peut être exécutée plus tard, ou sur un autre thread (séance 4). Si elle pouvait modifier la variable, on aurait des résultats imprévisibles. La bonne réponse n'est pas de contourner la règle (avec un tableau `int[1]` ou un `AtomicInteger`), mais de **laisser l'API calculer** :

```
// Fichier : LambdaCapture.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
void main() {
    var orders = SampleData.platform().getOrders();

    // ✅ On lit une variable effectivement finale...
    OrderStatus wanted = OrderStatus.PENDING;
    // ...et on laisse le stream compter (section 2)
    long pendingCount = orders.stream()
                              .filter(o -> o.getStatus() == wanted)
                              .count();
    IO.println(pendingCount); // 1
}
```

1.6 🟢 Les interfaces fonctionnelles du JDK

Vous n'aurez presque jamais besoin d'écrire vos propres interfaces fonctionnelles : le paquet `java.util.function` en fournit une quarantaine. Cinq suffisent pour 90 % des usages :

Interface	Méthode abstraite	Rôle	Exemple FoodFast
`Predicate<T>`	`boolean test(T t)`	**tester** une condition	la commande est-elle en attente ?

Interface	Méthode abstraite	Rôle	Exemple FoodFast
`Function<T, R>`	`R apply(T t)`	**transformer** un `T` en `R`	commande → son total
`Consumer<T>`	`void accept(T t)`	**agir** sur un élément	notifier le client
`Supplier<T>`	`T get()`	**fournir** une valeur	générer un identifiant
`Comparator<T>` (`java.util`)	`int compare(T a, T b)`	**comparer** deux éléments	trier par total

Les `<T>` et `<R>` sont des **paramètres de type**, comme dans `List<Dish>` : `Function<Order, BigDecimal>` se lit « une fonction qui prend une `Order` et rend un `BigDecimal` ».

```java
// Fichier : FunctionalInterfaces.java (avec SampleData.java,
DeliveryPlatform.java, Order.java, OrderStatus.java,
OrderNotFoundException.java, Customer.java, User.java, Dish.java,
DishSize.java)
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;
import java.util.function.Consumer;
import java.util.function.Function;
import java.util.function.Predicate;
import java.util.function.Supplier;

void main() {
    List<Order> orders = new ArrayList<>(SampleData.platform().getOrders());

    Predicate<Order> isPending = o -> o.getStatus() == OrderStatus.PENDING;
    Function<Order, BigDecimal> toTotal = o -> o.getTotal();
    Consumer<Order> notify = o -> IO.println("Notification envoyée pour " +
o.getStatus());
    Supplier<String> ticketNumber = () -> "T-" + System.nanoTime();
    Comparator<Order> byTotal = (a, b) -> a.getTotal().compareTo(b.getTotal());

    orders.sort(byTotal);                       // List.sort attend un
Comparator
    Order cheapest = orders.getFirst();

    IO.println(toTotal.apply(cheapest));        // 9.80
    IO.println(isPending.test(cheapest));       // true
    notify.accept(cheapest);                    // Notification envoyée pour
PENDING
    IO.println(ticketNumber.get().startsWith("T-")); // true
    IO.println(orders.getLast().getTotal());    // 37.50
}
```

Remarquez `orders.sort(byTotal)` : la méthode `sort` de `List` (que vous connaissez) attend un `Comparator`. Avant les lambdas, il fallait une classe anonyme ; aujourd'hui, une ligne suffit.

🔵 **`Discount` existe déjà dans le JDK.** Une fonction qui prend un `BigDecimal` et rend un `BigDecimal`, c'est un `Function<BigDecimal, BigDecimal>`, ou plus précisément un `UnaryOperator<BigDecimal>` (une fonction de `T` vers `T`). Garder notre propre interface `Discount` reste un bon choix : son nom dit **ce qu'elle signifie** dans le métier.

🔵 **Composer des comportements.** Ces interfaces ont des méthodes `default` (vues en séance 2) qui permettent de les combiner :

```java
// Fichier : Composition.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.math.BigDecimal;
import java.util.Comparator;
import java.util.function.Predicate;

void main() {
    var orders = SampleData.platform().getOrders();

    Predicate<Order> isDelivered = o -> o.getStatus() == OrderStatus.DELIVERED;
    Predicate<Order> isBig = o -> o.getTotal().compareTo(new BigDecimal("25"))
>= 0;

    // and, or, negate
    long bigDelivered = orders.stream().filter(isDelivered.and(isBig)).count();
    long notDelivered = orders.stream().filter(isDelivered.negate()).count();
    IO.println(bigDelivered + " " + notDelivered); // 1 3

    // Comparator.comparing, thenComparing, reversed
    Comparator<Order> byCustomerThenTotal =
        Comparator.comparing((Order o) -> o.getCustomer().getName())
                  .thenComparing(Order::getTotal, Comparator.reverseOrder());
    orders.stream()
          .sorted(byCustomerThenTotal)
          .forEach(o -> IO.println(o.getCustomer().getName() + " " +
o.getTotal()));
    // Alice 30.50
    // Alice 22.90
    // Alice 9.80
    // Bob 37.50
    // Bob 23.80
    // Bob 17.40
}
```

> 🔒 **Sécurité** — Les `Predicate` composables sont un bon outil pour écrire des **règles de validation** lisibles et testables une par une (`isNotBlank.and(hasValidLength).and(matchesFormat)`). Une règle = un prédicat = un test unitaire.

1.7 🟢 Les références de méthodes (et enfin `Integer::sum`)

Souvent, une lambda ne fait **qu'appeler une méthode qui existe déjà** : `o -> o.getTotal()`, `(a, b) -> a.add(b)`. Java propose alors une écriture encore plus courte, la

référence de méthode, avec `::`.

Forme	Lambda équivalente	Exemple
`Classe::méthodeStatique`	`(a, b) -> Classe.méthodeStatique(a, b)`	`Integer::sum`
`Classe::méthodeDInstance`	`o -> o.méthode()` (le 1er paramètre devient l'objet)	`Order::getTotal`, `BigDecimal::add`
`objet::méthode`	`x -> objet.méthode(x)`	`platform::getOrderById`
`Classe::new`	`() -> new Classe()`	`ArrayList::new`

`Integer::sum`, expliqué pour de bon. En séance 2, vous avez écrit dans `Order.addDish` :

```
items.merge(dish, quantity, Integer::sum);
```

`Map.merge(clé, valeur, fonction)` fonctionne ainsi : si la clé est absente, il range la valeur ; si elle est présente, il appelle `fonction(ancienneValeur, valeur)` et range le résultat. Le troisième paramètre est donc une fonction qui prend **deux `Integer` et en rend un** : un `BiFunction<Integer, Integer, Integer>`. Or la classe `Integer` possède justement une méthode statique `public static int sum(int a, int b)`. Les trois écritures suivantes sont équivalentes :

```java
// Fichier : MethodReferences.java (avec SampleData.java,
DeliveryPlatform.java, Order.java, OrderStatus.java,
OrderNotFoundException.java, Customer.java, User.java, Dish.java,
DishSize.java)
import java.math.BigDecimal;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.function.Function;

void main() {
    Map<String, Integer> cart = new HashMap<>();
    cart.merge("Margherita", 2, (oldQty, newQty) -> oldQty + newQty);    //
lambda
    cart.merge("Margherita", 1, (a, b) -> Integer.sum(a, b));             //
lambda qui appelle sum
    cart.merge("Margherita", 1, Integer::sum);                            //
référence de méthode
    IO.println(cart); // {Margherita=4}

    // Méthode d'instance d'un objet quelconque : le paramètre devient l'objet
    Function<Order, BigDecimal> toTotal = Order::getTotal;   // o ->
o.getTotal()

    // Méthode d'instance d'un objet précis
    DeliveryPlatform platform = SampleData.platform();
    String someId = platform.getOrders().iterator().next().getId();
    Function<String, Order> lookup = platform::getOrderById; // id ->
platform.getOrderById(id)
    IO.println(toTotal.apply(lookup.apply(someId)).signum()); // 1

    // Deux paramètres, le premier devient l'objet : (a, b) -> a.add(b)
    BigDecimal sum = List.of(new BigDecimal("1.50"), new BigDecimal("2.50"))
                         .stream()
                         .reduce(BigDecimal.ZERO, BigDecimal::add);
    IO.println(sum); // 4.00
}
```

Quand préférer la référence de méthode ? Quand elle est plus lisible que la lambda. `Order::getTotal` se lit mieux que `o -> o.getTotal()`. En revanche, `o -> o.getStatus() == OrderStatus.PENDING` n'a pas d'équivalent en référence de méthode, et c'est très bien ainsi : n'inventez pas une méthode juste pour pouvoir écrire `::`.

---

2. L'API Stream

2.1 🟢 Pourquoi ? Dire *quoi*, pas *comment*

Le problème. « Donner les identifiants des commandes en attente, de la plus chère à la moins chère. » Avec une boucle, il faut écrire la mécanique : une liste temporaire, un `if`, un tri, une deuxième boucle pour extraire les identifiants.

```
// Fichier : ImperativeVsStream.java (avec SampleData.java,
DeliveryPlatform.java, Order.java, OrderStatus.java,
OrderNotFoundException.java, Customer.java, User.java, Dish.java,
DishSize.java)
import java.math.BigDecimal;
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

void main() {
    var orders = SampleData.platform().getOrders();

    // Style impératif : on décrit COMMENT faire
    List<BigDecimal> totals1 = new ArrayList<>();
    for (Order o : orders) {
        if (o.getStatus() != OrderStatus.CANCELLED) {
            totals1.add(o.getTotal());
        }
    }
    totals1.sort(Comparator.reverseOrder());

    // Style déclaratif (Stream) : on décrit QUOI obtenir
    List<BigDecimal> totals2 = orders.stream()
            .filter(o -> o.getStatus() != OrderStatus.CANCELLED)
            .map(Order::getTotal)
            .sorted(Comparator.reverseOrder())
            .toList();

    IO.println(totals1); // [37.50, 30.50, 23.80, 17.40, 9.80]
    IO.println(totals2); // [37.50, 30.50, 23.80, 17.40, 9.80]
}
```

La version Stream se lit comme la phrase de départ : « garder les commandes non annulées, prendre leur total, trier par ordre décroissant, en faire une liste ». Pas de liste temporaire remplie à la main, pas d'état intermédiaire à surveiller.

> Une boucle `for` n'est pas « interdite » pour autant : pour un traitement avec beaucoup de branches ou un `break` précoce, elle reste souvent plus claire. Le stream brille pour **filtrer, transformer, agréger**.

2.2 🟢 Anatomie d'un pipeline

Un stream n'est **pas** une collection : il ne stocke rien. C'est un **tuyau** dans lequel les éléments d'une source passent un par un.

```
  source            opérations intermédiaires          opération terminale
orders.stream()  →  .filter(...)  →  .map(...)  →  ...  →  .toList()
  (Collection)      (rendent un Stream, paresseuses)       (déclenche le
calcul,
                                                            rend un résultat)
```

1. **Une source** : `collection.stream()`, `Stream.of(a, b, c)`, `Arrays.stream(tableau)`, `Files.lines(chemin)`…
2. **Zéro ou plusieurs opérations intermédiaires** : `filter`, `map`, `sorted`, `limit`, `distinct`… Chacune rend un nouveau `Stream`. Elles sont **paresseuses** : rien ne s'exécute tant qu'il n'y a pas d'opération terminale (voir 2.8).
3. **Une opération terminale** : `toList`, `count`, `anyMatch`, `reduce`, `collect`, `forEach`… Elle déclenche le calcul et **ferme** le stream.

Un stream **ne modifie jamais sa source** : `filter` ne retire rien de la `Map` des commandes, il produit un nouveau flux.

2.3 🟢 `filter`, `map`, `toList`

- `filter(Predicate<T>)` : garde les éléments pour lesquels le prédicat rend `true`.
- `map(Function<T, R>)` : transforme chaque élément (un `Stream<Order>` devient un `Stream<BigDecimal>`, par exemple).
- `toList()` : rassemble les éléments dans une `List`.

C'est avec ces trois opérations qu'on écrit les méthodes de recherche de `DeliveryPlatform` (annexe A.8) :

```java
public List<Order> findOrdersByStatus(OrderStatus status) {
    return orders.values().stream()
                 .filter(o -> o.getStatus() == status)
                 .toList();
}
```

Exemple complet :

```java
// Fichier : FilterMap.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.util.List;

void main() {
    DeliveryPlatform platform = SampleData.platform();

    // filter : les commandes livrées
    List<Order> delivered = platform.findOrdersByStatus(OrderStatus.DELIVERED);
    IO.println(delivered.size()); // 3

    // filter + map : les noms des clients qui ont une commande en cours de
livraison
    List<String> names = platform.getOrders().stream()
            .filter(o -> o.getStatus() == OrderStatus.IN_DELIVERY)
            .map(o -> o.getCustomer().getName())
            .toList();
    IO.println(names); // [Bob]

    // map sur un menu : List<Dish> -> List<String>
    List<Dish> menu = List.of(SampleData.MARGHERITA, SampleData.BURGER,
SampleData.SALADE);
    List<String> labels = menu.stream()
            .map(d -> d.name() + " (" + d.price() + " €)")
            .toList();
    IO.println(labels); // [Margherita (12.50 €), Burger (11.90 €), Salade
César (9.80 €)]
}
```

Piège : la liste rendue par `toList()` est non modifiable. Depuis Java 16, `stream.toList()` remplace l'ancien `collect(Collectors.toList())`. Il est plus court, mais il rend une liste **non modifiable** : toute tentative d'ajout ou de suppression lève une `UnsupportedOperationException`. C'est le plus souvent ce qu'on veut (on retrouve l'idée de copie défensive de la séance 2 : l'appelant ne peut pas modifier vos données).

```java
// Fichier : ToListUnmodifiable.java (exception attendue)
import java.util.List;

void main() {
    List<String> dishes = List.of("Margherita", "Burger").stream()
                               .map(String::toUpperCase)
                               .toList();
    dishes.add("Tiramisu"); // ❌ UnsupportedOperationException
}
```

✅ Si vous avez besoin d'une liste modifiable, demandez-la explicitement :

```java
List<String> editable = new ArrayList<>(stream.toList());
// ou : stream.collect(Collectors.toCollection(ArrayList::new));
```

2.4 🟢 `sorted` et `limit` : le top 3 des commandes

- `sorted(Comparator<T>)` : trie les éléments selon le comparateur.
- `limit(n)` : ne garde que les `n` premiers éléments.

Pour construire le comparateur, `Comparator.comparing(fonction)` est l'outil standard : on lui donne la **clé de tri**, il fabrique le `Comparator`. `.reversed()` inverse l'ordre.

```java
// Fichier : TopOrders.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.util.Comparator;
import java.util.List;

void main() {
    var orders = SampleData.platform().getOrders();

    List<Order> top3 = orders.stream()
            .filter(o -> o.getStatus() != OrderStatus.CANCELLED)
            .sorted(Comparator.comparing(Order::getTotal).reversed())
            .limit(3)
            .toList();

    top3.forEach(o -> IO.println(o.getCustomer().getName() + " : " +
o.getTotal()));
    // Bob : 37.50
    // Alice : 30.50
    // Bob : 23.80

    // Les commandes les plus récentes d'abord
    List<Order> recent = orders.stream()
            .sorted(Comparator.comparing(Order::getCreatedAt).reversed())
            .limit(2)
            .toList();
    IO.println(recent.size()); // 2
}
```

`Comparator.comparing(Order::getTotal)` fonctionne parce que `BigDecimal` implémente `Comparable` : il est comparé avec `compareTo`, comme il se doit pour un montant (séance 2).

2.5 🟢 `count`, `anyMatch`, `allMatch`, `findFirst`

Ces opérations terminales répondent à des questions :

Opération	Question	Type rendu
`count()`	combien ?	`long`
`anyMatch(p)`	au moins un ?	`boolean`
`allMatch(p)`	tous ?	`boolean`
`noneMatch(p)`	aucun ?	`boolean`
`findFirst()`	le premier (s'il existe)	`Optional<T>`

```java
// Fichier : Matching.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.math.BigDecimal;
import java.util.Optional;

void main() {
    var orders = SampleData.platform().getOrders();

    long pending = orders.stream().filter(o -> o.getStatus() ==
OrderStatus.PENDING).count();
    boolean anyBig = orders.stream().anyMatch(o -> o.getTotal().compareTo(new
BigDecimal("35")) > 0);
    boolean allPaid = orders.stream().allMatch(o -> o.getTotal().signum() > 0);
    boolean noEmpty = orders.stream().noneMatch(o -> o.getItems().isEmpty());

    IO.println(pending + " " + anyBig + " " + allPaid + " " + noEmpty); // 1
true true true

    // findFirst rend un Optional : il peut ne rien y avoir (séance 2)
    Optional<Order> preparing = orders.stream()
            .filter(o -> o.getStatus() == OrderStatus.PREPARING)
            .findFirst();
    IO.println(preparing.isPresent()); // false
}
```

`anyMatch`, `allMatch`, `noneMatch` et `findFirst` sont **court-circuitants** : ils s'arrêtent dès que la réponse est connue, sans parcourir tout le flux.

2.6 🟢 Chiffre d'affaires : `reduce` avec `BigDecimal`

Le problème. Pour les entiers et les `double`, les streams spécialisés offrent `sum()` : `IntStream.of(1, 2, 3).sum()`. Mais il n'existe pas de `sum()` pour `BigDecimal`, et on n'utilise **jamais** `double` pour de l'argent (séance 2).

La solution : `reduce`. `reduce(identité, accumulateur)` combine les éléments deux à deux :

- **l'identité** est la valeur de départ, et le résultat si le flux est vide : `BigDecimal.ZERO` ;
- **l'accumulateur** dit comment combiner le résultat partiel et l'élément suivant : `BigDecimal::add`, c'est-à-dire `(acc, x) -> acc.add(x)`.

```
reduce(ZERO, add) sur [30.50, 17.40, 23.80]
  ZERO.add(30.50)  = 30.50
  30.50.add(17.40) = 47.90
  47.90.add(23.80) = 71.70
```

```java
// Fichier : Revenue.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;

void main() {
    var orders = SampleData.platform().getOrders();

    // Chiffre d'affaires : total des commandes livrées
    BigDecimal revenue = orders.stream()
            .filter(o -> o.getStatus() == OrderStatus.DELIVERED)
            .map(Order::getTotal)
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    IO.println("CA : " + revenue); // CA : 71.70

    // Sur un flux vide, on obtient l'identité, pas une erreur
    BigDecimal none = List.<BigDecimal>of().stream().reduce(BigDecimal.ZERO,
BigDecimal::add);
    IO.println(none); // 0

    // Le total d'une commande, version stream (même résultat que
Order.getTotal)
    Order first =
SampleData.platform().findOrdersByStatus(OrderStatus.PENDING).getFirst();
    BigDecimal total = first.getItems().entrySet().stream()
            .map(e ->
e.getKey().price().multiply(BigDecimal.valueOf(e.getValue())))
            .reduce(BigDecimal.ZERO, BigDecimal::add);
    IO.println(total.compareTo(first.getTotal()) == 0); // true
}
```

Piège : `BigDecimal` est immuable. `acc.add(x)` ne modifie pas `acc`, il rend un **nouveau** `BigDecimal`. C'est pour cela que `reduce` convient parfaitement, alors qu'une boucle `forEach(o -> revenue.add(...))` ne ferait rien d'utile (et ne compilerait même pas si `revenue` était réaffectée : voir 1.5).

🔵 **La version à un argument.** `reduce(BigDecimal::add)` (sans identité) rend un `Optional<BigDecimal>`, vide si le flux est vide. Pour un chiffre d'affaires, zéro est la bonne réponse sur un flux vide : préférez la version avec identité.

2.7 🟢 Grouper : `Collectors.groupingBy` et `counting`

Le problème. « Combien de commandes par statut ? » Avec une boucle, c'est un `Map.merge` par élément. Avec un stream, c'est l'opération terminale `collect`, qui reçoit un **collecteur** de la classe `Collectors`.

`Collectors.groupingBy(classifieur, collecteurAval)` :

- le **classifieur** (une `Function`) calcule la clé de chaque élément : ici, son statut ;
- le **collecteur aval** dit quoi faire des éléments de chaque groupe : les compter (`counting()`), les additionner (`summingInt(...)`), les mettre en liste (par défaut)…

```java
// Fichier : OrdersByStatus.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.math.BigDecimal;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

void main() {
    var orders = SampleData.platform().getOrders();

    // Nombre de commandes par statut
    Map<OrderStatus, Long> countByStatus = orders.stream()
            .collect(Collectors.groupingBy(Order::getStatus,
Collectors.counting()));
    IO.println(countByStatus.get(OrderStatus.DELIVERED)); // 3
    IO.println(countByStatus.get(OrderStatus.PREPARING)); // null : aucun
groupe vide n'est créé

    // Sans collecteur aval : les commandes elles-mêmes, en liste
    Map<String, List<Order>> byCustomer = orders.stream()
            .collect(Collectors.groupingBy(o -> o.getCustomer().getName()));
    IO.println(byCustomer.get("Alice").size()); // 3

    // Chiffre d'affaires par client : groupingBy + reducing
    Map<String, BigDecimal> revenueByCustomer = orders.stream()
            .filter(o -> o.getStatus() == OrderStatus.DELIVERED)
            .collect(Collectors.groupingBy(
                    o -> o.getCustomer().getName(),
                    Collectors.reducing(BigDecimal.ZERO, Order::getTotal,
BigDecimal::add)));
    IO.println(revenueByCustomer.get("Bob")); // 41.20

    // 🔵 Ordre garanti : demander une EnumMap (ordre de déclaration de l'enum)
    Map<OrderStatus, Long> ordered = orders.stream()
            .collect(Collectors.groupingBy(Order::getStatus,
                    () -> new EnumMap<>(OrderStatus.class),
Collectors.counting()));
    IO.println(ordered); // {PENDING=1, IN_DELIVERY=1, DELIVERED=3,
CANCELLED=1}
}
```

Piège : l'ordre d'une `Map` issue de `groupingBy` n'est pas garanti. Par défaut, `groupingBy` produit une `HashMap`. Si vous affichez `countByStatus`, l'ordre des clés peut changer d'une exécution à l'autre. N'écrivez jamais un test qui compare l'affichage d'une

`HashMap` à une chaîne : comparez les valeurs avec `get`, ou demandez une `EnumMap` / `TreeMap` comme ci-dessus.

2.8 🟢 Les trois pièges des streams

Piège 1 : un stream est paresseux

🤔 **Prédisez la sortie** :

```
// Fichier : LazyStream.java
import java.util.List;
import java.util.stream.Stream;

void main() {
    List<String> statuses = List.of("PENDING", "DELIVERED", "CANCELLED");

    Stream<String> s = statuses.stream()
            .filter(st -> {
                IO.println("filtre " + st);
                return st.startsWith("D");
            });
    IO.println("pipeline construit");

    List<String> result = s.toList();
    IO.println(result);
}
```

Réponse :

```
pipeline construit
filtre PENDING
filtre DELIVERED
filtre CANCELLED
[DELIVERED]
```

Rien n'est filtré tant que `toList()` n'est pas appelé. Les opérations intermédiaires **décrivent** un calcul ; seule l'opération terminale l'**exécute**. Conséquence pratique : un pipeline sans opération terminale ne fait **rien**, sans erreur ni avertissement.

```
// ❌ Oubli de l'opération terminale : aucune commande n'est annulée
orders.stream().filter(o -> o.getStatus() == OrderStatus.PENDING)
               .map(o -> { o.moveTo(OrderStatus.CANCELLED); return o; });
```

(Ce code cumule deux défauts : pas d'opération terminale, et un effet de bord dans `map`. Voir le piège 3.)

Piège 2 : un stream ne sert qu'une fois

```
// Fichier : StreamReuse.java (exception attendue)
import java.util.List;
import java.util.stream.Stream;

void main() {
    Stream<String> dishes = List.of("Margherita", "Burger").stream();
    IO.println(dishes.count());   // 2
    IO.println(dishes.toList());  // ❌ IllegalStateException:
                                  //    stream has already been operated upon
or closed
}
```

✅ Ne stockez pas un `Stream` dans une variable ou un attribut : stockez la **collection**, et appelez `.stream()` à chaque fois que vous en avez besoin.

Piège 3 : pas d'effets de bord

Une lambda passée à `filter`, `map` ou `sorted` doit être une **fonction pure** : elle calcule un résultat à partir de son paramètre, sans modifier quoi que ce soit à l'extérieur.

```
// ❌ Effet de bord : on remplit une liste externe depuis forEach
List<Order> pending = new ArrayList<>();
orders.stream()
      .filter(o -> o.getStatus() == OrderStatus.PENDING)
      .forEach(o -> pending.add(o));   // fragile, et faux en parallèle (2.11)

// ✅ On laisse le stream construire le résultat
List<Order> pending = orders.stream()
      .filter(o -> o.getStatus() == OrderStatus.PENDING)
      .toList();
```

`forEach` a sa place en **fin** de pipeline, pour une action réellement extérieure : afficher, envoyer une notification. Pas pour construire un résultat.

2.9 🔵 `flatMap` : les plats les plus commandés

Le problème. Chaque commande contient une `Map<Dish, Integer>` de plats. Pour savoir quels plats sont les plus commandés sur toute la plateforme, il faut « aplatir » : passer d'un

flux de commandes à un flux de lignes (plat, quantité).

`map` transforme **un** élément en **un** élément. `flatMap` transforme **un** élément en **un flux** d'éléments, puis concatène tous ces flux.

```
map     : Order  ->  Stream<Map.Entry<Dish, Integer>>     =>
Stream<Stream<...>>  (pas pratique)
flatMap : Order  ->  Stream<Map.Entry<Dish, Integer>>     =>
Stream<Map.Entry<...>> (à plat)
```

```
// Fichier : TopDishes.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.util.Map;
import java.util.stream.Collectors;

void main() {
    var orders = SampleData.platform().getOrders();

    Map<String, Integer> quantityByDish = orders.stream()
            .filter(o -> o.getStatus() != OrderStatus.CANCELLED)
            .flatMap(o -> o.getItems().entrySet().stream())   // une ligne par
(plat, quantité)
            .collect(Collectors.groupingBy(
                    e -> e.getKey().name(),
                    Collectors.summingInt(Map.Entry::getValue)));

    quantityByDish.entrySet().stream()
            .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
            .limit(3)
            .forEach(e -> IO.println(e.getKey() + " x" + e.getValue()));
    // Margherita x5
    // Burger x3
    // Tiramisu x2
}
```

2.10 🔵 `Collectors.toMap` et les collisions de clés

`Collectors.toMap(fonctionClé, fonctionValeur)` construit une `Map` à partir d'un flux. Pratique pour indexer un menu par nom :

```
Map<String, BigDecimal> priceByName = menu.stream()
        .collect(Collectors.toMap(Dish::name, Dish::price));
```

Piège : deux éléments avec la même clé. Si deux plats portent le même nom (une Margherita `MEDIUM` et une Margherita `LARGE`), `toMap` ne sait pas laquelle garder et lève une `IllegalStateException` :

```
// Fichier : ToMapCollision.java (exception attendue)
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

void main() {
    List<Dish> menu = List.of(
            new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM),
            new Dish("Margherita", new BigDecimal("15.90"), DishSize.LARGE));

    // ❌ IllegalStateException: Duplicate key Margherita (attempted merging
values 12.50 and 15.90)
    Map<String, BigDecimal> priceByName = menu.stream()
            .collect(Collectors.toMap(Dish::name, Dish::price));
}
```

✅ Deux solutions, selon le besoin métier :

```java
// Fichier : ToMapMerge.java
import java.math.BigDecimal;
import java.util.List;
import java.util.Map;
import java.util.stream.Collectors;

enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

void main() {
    List<Dish> menu = List.of(
            new Dish("Margherita", new BigDecimal("12.50"), DishSize.MEDIUM),
            new Dish("Margherita", new BigDecimal("15.90"), DishSize.LARGE));

    // 1. Une fonction de fusion qui décide : ici, garder le prix le plus bas
("à partir de")
    Map<String, BigDecimal> fromPrice = menu.stream()
            .collect(Collectors.toMap(Dish::name, Dish::price,
BigDecimal::min));
    IO.println(fromPrice); // {Margherita=12.50}

    // 2. Ou garder tous les plats de même nom : groupingBy
    Map<String, List<Dish>> byName = menu.stream()
            .collect(Collectors.groupingBy(Dish::name));
    IO.println(byName.get("Margherita").size()); // 2
}
```

Autre piège de `toMap` : une **valeur** `null` lève une `NullPointerException` (contrairement à `HashMap.put`). Les records FoodFast refusent déjà `null` dans leur constructeur, ce qui vous protège.

2.11 🔵 Streams parallèles : pourquoi les éviter par défaut

Remplacer `stream()` par `parallelStream()` découpe le travail entre plusieurs cœurs. C'est tentant, mais **rarement une bonne idée** dans une application métier :

- **Ce n'est pas plus rapide sur de petites collections.** Découper, répartir et fusionner a un coût. Sur quelques milliers de commandes, le stream séquentiel gagne presque toujours. Le gain n'apparaît que sur de gros volumes avec un calcul coûteux par élément, et se **mesure** (avec un outil comme JMH), il ne se devine pas.
- **Les effets de bord deviennent des bugs.** Le `forEach(pending::add)` du piège 3 sur une `ArrayList`, en parallèle, perd des éléments ou lève une exception, de façon aléatoire : c'est une *race condition* (séance 4).
- **L'ordre n'est plus garanti** avec `forEach` (il faut `forEachOrdered`).

- Tous les streams parallèles partagent par défaut le même pool de threads (`ForkJoinPool.commonPool()`) : un traitement lent bloque les autres.

```
// ❌ Résultat imprévisible : ArrayList n'est pas faite pour être remplie par plusieurs threads
List<Integer> ids = new ArrayList<>();
IntStream.range(0, 100_000).parallel().forEach(ids::add);
IO.println(ids.size()); // souvent < 100000, parfois ArrayIndexOutOfBoundsException

// ✅ Sans effet de bord, le parallélisme est correct (mais pas forcément plus rapide)
List<Integer> ids = IntStream.range(0, 100_000).parallel().boxed().toList();
```

> Règle : `stream()` par défaut. `parallelStream()` seulement après avoir mesuré un gain réel, et avec des lambdas sans effet de bord.

2.12 🟣 Gatherers (JDK 24) : des opérations intermédiaires sur mesure

L'API Stream a un nombre fixe d'opérations intermédiaires. Depuis le JDK 24 (JEP 485, fonctionnalité finale), la méthode `gather(...)` permet d'en brancher de nouvelles. La classe `Gatherers` en fournit quelques-unes, prêtes à l'emploi :

Gatherer	Effet
`windowFixed(n)`	découpe en lots de `n` éléments (le dernier peut être plus petit)
`windowSliding(n)`	fenêtres glissantes de `n` éléments
`scan(init, f)`	cumul progressif (chaque étape intermédiaire est émise)
`fold(init, f)`	cumul final (un seul élément émis)

Exemple FoodFast : un livreur emporte au plus deux commandes par tournée. On veut découper les commandes prêtes en lots de deux, et suivre le chiffre d'affaires cumulé.

```java
// Fichier : DeliveryBatches.java
import java.math.BigDecimal;
import java.util.List;
import java.util.stream.Gatherers;

void main() {
    List<String> ready = List.of("order-1", "order-2", "order-3", "order-4",
"order-5");

    List<List<String>> rounds = ready.stream()
            .gather(Gatherers.windowFixed(2))
            .toList();
    IO.println(rounds); // [[order-1, order-2], [order-3, order-4], [order-5]]

    List<BigDecimal> totals = List.of(new BigDecimal("30.50"), new
BigDecimal("17.40"),
                                      new BigDecimal("23.80"));
    List<BigDecimal> runningRevenue = totals.stream()
            .gather(Gatherers.scan(() -> BigDecimal.ZERO, BigDecimal::add))
            .toList();
    IO.println(runningRevenue); // [30.50, 47.90, 71.70]
}
```

Pour aller plus loin, on peut écrire son propre `Gatherer` (par exemple « dédoublonner selon une clé » ou « limiter le débit ») : voir la JEP 485.

---

3. Les exceptions

3.1 🟢 Pourquoi ? Une erreur doit se voir

Une application de livraison rencontre des erreurs en permanence : un identifiant de commande qui n'existe pas, une quantité saisie « abc », un fichier de menu absent, une transition de statut interdite. La question n'est pas *s'il y aura des erreurs*, mais **comment elles sont signalées**.

Avant les exceptions, on signalait une erreur par une **valeur spéciale** : `-1`, `0`, `null`. C'est dangereux, parce que l'appelant peut l'ignorer sans s'en rendre compte :

```
// ❌ Valeur magique : rien n'oblige l'appelant à vérifier
static int parseQuantity(String input) {
    try {
        return Integer.parseInt(input);
    } catch (NumberFormatException e) {
        return 0;   // "abc" devient 0 : la commande part avec 0 pizza, ou plus
tard
    }               // une IllegalArgumentException surgit loin de la vraie
cause
}
```

Une **exception** interrompt le flux normal et **remonte la pile d'appels** jusqu'à ce que quelqu'un la traite. Si personne ne la traite, le programme s'arrête avec un message explicite. Une erreur ne peut donc pas passer inaperçue. C'est le principe *fail fast* que vous avez appliqué en séance 2 dans les constructeurs (`Dish` refuse un prix négatif).

3.2 🟢 La hiérarchie : checked ou unchecked

Toutes les exceptions héritent de `Throwable` :

```
Throwable
├── Error                       problème grave de la JVM : on ne l'attrape pas
│   └── OutOfMemoryError, StackOverflowError…
└── Exception                   CHECKED : le compilateur oblige à la traiter
    ├── IOException, SQLException…
    └── RuntimeException        UNCHECKED : aucune obligation
        ├── IllegalArgumentException   (paramètre invalide)
        │   └── NumberFormatException
        ├── IllegalStateException      (opération interdite dans l'état actuel)
        ├── NullPointerException
        ├── UnsupportedOperationException
        └── OrderNotFoundException     (la nôtre, section 3.5)
```

Checked (vérifiée). Toute sous-classe d'`Exception` qui n'est pas une `RuntimeException`. Le compilateur **exige** que vous la traitiez (`try` / `catch`) ou que vous la déclariez (`throws`). Elle représente un problème **extérieur au programme**, prévisible, contre lequel le code ne peut rien : fichier absent, réseau coupé, base de données indisponible.

```
// ❌ Ne compile pas : unreported exception IOException; must be caught or
declared to be thrown
String loadMenu(Path file) {
    return Files.readString(file);
}

// ✅ Option 1 : déclarer, et laisser l'appelant décider
String loadMenu(Path file) throws IOException {
    return Files.readString(file);
}
```

Unchecked (non vérifiée). `RuntimeException` et ses sous-classes. Aucune obligation de traitement. Elle signale le plus souvent une **erreur de programmation** ou une **violation de contrat** : un paramètre invalide, une méthode appelée au mauvais moment, un `null` inattendu. La bonne réaction est en général de corriger l'appelant, pas d'attraper l'exception.

	Checked	**Unchecked**
Hérite de	`Exception`	`RuntimeException`
Le compilateur oblige à la traiter	oui	non
Cause typique	monde extérieur (I/O, réseau)	bug, contrat violé, règle métier
Exemples	`IOException`, `SQLException`	`IllegalArgumentException`, `IllegalStateException`

`throw` et `throws`, à ne pas confondre :

- `throw new X(...)` est une **instruction** : elle lève l'exception, maintenant.
- `throws X` est une **déclaration** dans la signature : « cette méthode peut laisser sortir un `X` ».

3.3 🟢 `try` / `catch` / `finally` et le multi-catch

```
try {
    // code qui peut lever une exception
} catch (TypeA e) {
    // exécuté seulement si une TypeA (ou une sous-classe) est levée
} catch (TypeB | TypeC e) {
    // multi-catch : même traitement pour deux types
} finally {
    // TOUJOURS exécuté : succès, exception attrapée ou non
}
```

Où attraper ? Là où l'on **sait quoi faire** de l'erreur. Typiquement à la **frontière** du programme : là où une saisie utilisateur entre (on la refuse avec un message clair), là où l'on répond à une requête (on renvoie une erreur propre). Au milieu du code métier, on laisse le plus souvent remonter.

Exemple FoodFast : un client saisit une quantité et une date de livraison souhaitée. Les deux peuvent être mal formées.

```java
// Fichier : ParseInput.java
import java.time.LocalDateTime;
import java.time.format.DateTimeParseException;

record DeliveryRequest(int quantity, LocalDateTime wishedAt) {
    DeliveryRequest {
        if (quantity <= 0) throw new IllegalArgumentException("Quantité
invalide : " + quantity);
    }
}

/** Frontière : on transforme une saisie texte en objet métier, ou on la
refuse. */
static String handle(String quantityText, String dateText) {
    try {
        var request = new DeliveryRequest(Integer.parseInt(quantityText),
                                          LocalDateTime.parse(dateText));
        return "OK : " + request;
    } catch (NumberFormatException | DateTimeParseException e) {
        // multi-catch : saisie mal formée
        return "Saisie mal formée, vérifiez la quantité et la date.";
    } catch (IllegalArgumentException e) {
        // saisie bien formée, mais refusée par la règle métier
        return "Saisie refusée : la quantité doit être positive.";
    } finally {
        IO.println("(saisie traitée)");
    }
}

void main() {
    IO.println(handle("2", "2026-11-03T19:30"));
    IO.println(handle("deux", "2026-11-03T19:30"));
    IO.println(handle("-1", "2026-11-03T19:30"));
}
// (saisie traitée)
// OK : DeliveryRequest[quantity=2, wishedAt=2026-11-03T19:30]
// (saisie traitée)
// Saisie mal formée, vérifiez la quantité et la date.
// (saisie traitée)
// Saisie refusée : la quantité doit être positive.
```

Piège : l'ordre des `catch`. `NumberFormatException` est une sous-classe d'`IllegalArgumentException`. Si l'on met `catch (IllegalArgumentException e)` en premier, le bloc `NumberFormatException` devient inatteignable et le compilateur refuse le code (`exception NumberFormatException has already been caught`). Règle : **du plus spécifique au plus général**.

Piège : attraper trop large. `catch (Exception e)` attrape aussi les `NullPointerException` et autres bugs, qui seront alors traités comme une « saisie mal formée ». Attrapez les types **précis** que vous savez traiter.

Piège : `return` dans `finally`. Un `return` dans un bloc `finally` écrase le résultat du `try` et fait disparaître toute exception en cours. Ne mettez dans `finally` que du nettoyage, jamais de `return`.

3.4 🟢 try-with-resources

Le problème. Un fichier, une connexion réseau, une connexion à la base (séance 4) sont des **ressources** : elles doivent être fermées, même si une exception survient au milieu. Avec `finally`, c'est verbeux et facile à rater.

La solution. Toute classe qui implémente `AutoCloseable` peut être déclarée entre parenthèses après `try`. Java appelle `close()` automatiquement à la sortie du bloc, quoi qu'il arrive.

Exemple FoodFast : charger le menu depuis un fichier CSV. `Files.lines` rend un `Stream<String>` adossé au fichier ouvert : c'est l'un des rares streams qu'il **faut** fermer.

```java
// Fichier : MenuLoader.java
import java.io.IOException;
import java.math.BigDecimal;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.stream.Stream;

enum DishSize { SMALL, MEDIUM, LARGE }
record Dish(String name, BigDecimal price, DishSize size) {}

static Dish parseLine(String line) {
    String[] parts = line.split(";");
    return new Dish(parts[0], new BigDecimal(parts[1]),
DishSize.valueOf(parts[2]));
}

/** Lit le menu. IOException est checked : on la déclare, l'appelant décidera.
*/
static List<Dish> loadMenu(Path file) throws IOException {
    try (Stream<String> lines = Files.lines(file)) {   // fermé automatiquement
        return lines.filter(l -> !l.isBlank())
                    .map(l -> parseLine(l))
                    .toList();
    }
}

void main() throws IOException {
    Path file = Files.createTempFile("menu", ".csv");
    try {
        Files.writeString(file, """
                Margherita;12.50;MEDIUM
                Burger;11.90;LARGE
                Tiramisu;5.50;SMALL
                """);
        List<Dish> menu = loadMenu(file);
        IO.println(menu.size() + " plats, dont " + menu.getFirst().name()); //
3 plats, dont Margherita
    } finally {
        Files.deleteIfExists(file);   // nettoyage du fichier temporaire
    }
}
```

On peut déclarer plusieurs ressources, séparées par `;`. Elles sont fermées **dans l'ordre inverse** de leur ouverture.

🤔 **Prédisez la sortie** : dans quel ordre s'affichent les lignes ?

```java
// Fichier : CloseOrder.java
class Ticket implements AutoCloseable {
    Ticket() { IO.println("ouverture"); }
    @Override
    public void close() { IO.println("fermeture"); }
}

void main() {
    try (Ticket t = new Ticket()) {
        IO.println("préparation");
        throw new IllegalStateException("four en panne");
    } catch (IllegalStateException e) {
        IO.println("catch : " + e.getMessage());
    } finally {
        IO.println("finally");
    }
}
```

Réponse :

```
ouverture
préparation
fermeture
catch : four en panne
finally
```

La ressource est fermée **avant** l'entrée dans le `catch` : quand on traite l'erreur, le fichier ou la connexion est déjà libéré.

🔵 **Exceptions supprimées.** Si `close()` lève elle-même une exception alors qu'une exception est déjà en cours, la seconde n'écrase pas la première : elle lui est attachée et reste accessible avec `e.getSuppressed()`. Avec un `finally` écrit à la main, l'exception d'origine aurait été perdue.

3.5 🟢 Une exception métier : `OrderNotFoundException`

Le problème. `DeliveryPlatform.findOrderById` rend un `Optional<Order>` (séance 2). Mais dans beaucoup de cas, l'appelant **sait** que la commande doit exister : le livreur scanne le QR code d'une commande qu'on vient de lui attribuer, le client clique sur une commande de son historique. Si elle n'existe pas, ce n'est pas un cas normal à gérer : c'est une **anomalie**, et il faut un message qui dise *laquelle*.

Lever une `IllegalArgumentException("not found")` fonctionnerait, mais le type ne dit rien du problème. Une **exception métier** porte un nom du domaine et peut transporter des informations utiles :

```
/** Levée quand on exige une commande qui n'existe pas. */
class OrderNotFoundException extends RuntimeException {
    private final String orderId;

    OrderNotFoundException(String orderId) {
        super("Commande introuvable : " + orderId);
        this.orderId = orderId;
    }

    public String getOrderId() { return orderId; }
}
```

(Fichier complet en annexe A.7.)

Pourquoi unchecked ?

1. **L'appelant ne peut presque jamais réparer sur place.** Si la commande n'existe pas, le code métier n'a pas de plan B : l'erreur doit remonter jusqu'à la frontière (l'interface, l'API) qui la traduira en « commande introuvable » pour l'utilisateur. Une exception checked obligerait chaque méthode intermédiaire à écrire `throws OrderNotFoundException`, sans rien en faire.
2. **L'appelant peut l'éviter.** S'il n'est pas sûr que la commande existe, il dispose de `findOrderById` et de son `Optional` (section 3.6). Lever l'exception, c'est donc signaler que **le contrat de `getOrderById` n'a pas été respecté**, et c'est le rôle d'une `RuntimeException`.
3. **Les exceptions checked passent mal dans les lambdas** : `Function.apply` ne déclare aucune exception, donc `map(platform::getOrderById)` ne compilerait pas si `getOrderById` levait une exception checked (section 3.10).
4. C'est la pratique dominante en Java moderne : `IllegalArgumentException`, `IllegalStateException`, `NoSuchElementException` du JDK, et les exceptions métier des frameworks comme Spring, sont unchecked.

Et quand choisir checked ? Quand l'appelant **doit** prévoir un plan B et peut le mettre en œuvre lui-même : réessayer un appel réseau, proposer un autre fichier. Le JDK le fait pour les I/O (`IOException`).

Bonnes pratiques pour une exception métier :

- hériter du type le plus proche du sens : ici `RuntimeException` ; pour une règle violée, on pourrait hériter d'`IllegalStateException` ;
- un nom qui finit par `Exception` et qui parle métier ;
- un message qui aide **le développeur** à diagnostiquer (l'identifiant recherché) ; ce message est destiné aux logs, pas à l'utilisateur final (section 4.2) ;
- si elle enveloppe une autre exception, un constructeur `(String message, Throwable cause)` pour ne pas perdre la cause (section 3.9).

3.6 🟢 `getOrderById` ou `findOrderById` : lever ou renvoyer un `Optional` ?

`DeliveryPlatform` (annexe A.8) propose désormais les deux :

```
/** Recherche : l'absence est un résultat normal. */
public Optional<Order> findOrderById(String id) {
    return Optional.ofNullable(orders.get(id));
}

/** Accès : la commande DOIT exister, sinon c'est une anomalie. */
public Order getOrderById(String id) {
    return findOrderById(id).orElseThrow(() -> new OrderNotFoundException(id));
}
```

`orElseThrow(Supplier)` (vu en séance 2 sans argument) prend ici un `Supplier` : une lambda qui **fabrique** l'exception, seulement si l'`Optional` est vide.

Situation	Méthode	Pourquoi
Le client tape un numéro de commande dans une barre de recherche	`findOrderById`	ne rien trouver est un cas **normal**, on affiche « aucun résultat »
Le livreur valide la livraison d'une commande qui lui a été attribuée	`getOrderById`	la commande **doit** exister ; sinon il y a un bug ou une manipulation
Un traitement de fin de journée relit des identifiants stockés	`getOrderById`	une absence révèle une incohérence des données, il faut la voir

La règle : l'absence est-elle un résultat attendu ? Oui → `Optional`. Non → exception.

```java
// Fichier : GetOrFind.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
void main() {
    DeliveryPlatform platform = SampleData.platform();
    String knownId =
platform.findOrdersByStatus(OrderStatus.IN_DELIVERY).getFirst().getId();

    // Recherche : on gère les deux cas, sans exception
    platform.findOrderById("order-inconnue").ifPresentOrElse(
            o -> IO.println("Trouvée : " + o.getStatus()),
            () -> IO.println("Aucune commande ne correspond."));

    // Accès : la commande doit exister
    Order order = platform.getOrderById(knownId);
    order.moveTo(OrderStatus.DELIVERED);
    IO.println("Livrée : " + order.getStatus());

    // Anomalie : l'exception remonte avec un message précis
    try {
        platform.getOrderById("order-inconnue");
    } catch (OrderNotFoundException e) {
        IO.println("Anomalie : " + e.getMessage());
    }
}
// Aucune commande ne correspond.
// Livrée : DELIVERED
// Anomalie : Commande introuvable : order-inconnue
```

Pièges :

- ❌ `findOrderById(id).get()` : si l'`Optional` est vide, on obtient une `NoSuchElementException("No value present")`, qui ne dit pas quelle commande manquait. ✅ `getOrderById(id)`, ou `orElseThrow(() -> new OrderNotFoundException(id))`.
- ❌ Lever une exception pour un cas normal (recherche sans résultat) : les exceptions ne servent pas à piloter le flux normal du programme.
- ❌ Renvoyer `null` quand la commande n'existe pas : c'est précisément ce qu'`Optional` et `OrderNotFoundException` évitent.

3.7 🟢 Valider les transitions : `IllegalStateException`

Une commande suit un cycle de vie : `PENDING → CONFIRMED → PREPARING → IN_DELIVERY → DELIVERED`, avec une annulation possible tant qu'elle n'est pas partie. Passer de

`DELIVERED` à `PREPARING` n'a aucun sens. Si le code le permet, un bug (ou un utilisateur malveillant qui rejoue une requête) peut mettre les données dans un état incohérent.

La règle est portée par l'`enum` (annexe A.2), et `Order.moveTo` la fait respecter (annexe A.6) :

```
// Dans OrderStatus
boolean canMoveTo(OrderStatus next) {
    return switch (this) {
        case PENDING -> next == CONFIRMED || next == CANCELLED;
        case CONFIRMED -> next == PREPARING || next == CANCELLED;
        case PREPARING -> next == IN_DELIVERY || next == CANCELLED;
        case IN_DELIVERY -> next == DELIVERED;
        case DELIVERED, CANCELLED -> false;
    };
}

// Dans Order
public void moveTo(OrderStatus next) {
    Objects.requireNonNull(next, "next");
    if (!status.canMoveTo(next)) {
        throw new IllegalStateException("Transition interdite : " + status + "
-> " + next);
    }
    if (next == OrderStatus.CONFIRMED && items.isEmpty()) {
        throw new IllegalStateException("Commande vide");
    }
    status = next;
}
```

Le `switch` est **exhaustif** : si l'on ajoute un statut à l'`enum`, le compilateur signale le cas manquant. C'est une protection gratuite.

`IllegalArgumentException` ou `IllegalStateException` ?

- `IllegalArgumentException` : **le paramètre** est invalide en lui-même (quantité négative, nom vide).
- `IllegalStateException` : le paramètre est valide, mais **l'objet n'est pas dans le bon état** pour cette opération (livrer une commande annulée).

```
// Fichier : Transitions.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
void main() {
    Order order = new Order(SampleData.ALICE);
    order.addDish(SampleData.MARGHERITA, 1);
    order.moveTo(OrderStatus.CONFIRMED);
    order.moveTo(OrderStatus.CANCELLED);
    IO.println(order.getStatus()); // CANCELLED

    try {
        order.moveTo(OrderStatus.IN_DELIVERY); // on ne livre pas une commande
annulée
    } catch (IllegalStateException e) {
        IO.println(e.getMessage()); // Transition interdite : CANCELLED ->
IN_DELIVERY
    }
    IO.println(order.getStatus()); // CANCELLED : l'état n'a pas bougé
}
```

Point important : la vérification a lieu **avant** toute modification. Si la transition est refusée, l'objet reste dans un état cohérent. C'est le principe *fail-safe*.

3.8 🟢 Ne jamais avaler une exception, ni renvoyer `0` ou `null` « par défaut »

C'est l'erreur la plus fréquente, et la plus coûteuse : le programme continue avec des données fausses, et l'on découvre le problème des jours plus tard, loin de sa cause.

```java
// ❌ 1. Le catch vide : l'erreur disparaît sans laisser de trace
try {
    order.moveTo(OrderStatus.DELIVERED);
} catch (IllegalStateException e) {
    // rien
}

// ❌ 2. La valeur magique : la remise "invalide" devient 0 %, personne ne le saura
static int parsePercent(String input) {
    try {
        return Integer.parseInt(input);
    } catch (NumberFormatException e) {
        return 0;
    }
}

// ❌ 3. Le null "par défaut" : la NullPointerException surgira plus loin
static Order safeGet(DeliveryPlatform p, String id) {
    try {
        return p.getOrderById(id);
    } catch (OrderNotFoundException e) {
        return null;
    }
}

// ❌ 4. Le printStackTrace : la pile part sur la console, hors des logs,
//       et le programme continue comme si de rien n'était
catch (IOException e) {
    e.printStackTrace();
}
```

✅ Les bonnes réactions, par ordre de préférence :

1. **Ne pas attraper** : si vous ne savez pas quoi faire de l'erreur ici, laissez-la remonter.
2. **Valider en amont** plutôt qu'attraper : vérifiez l'entrée et levez une exception claire.
3. **Traiter vraiment** : refuser la saisie avec un message, annuler la commande, réessayer.
4. **Journaliser et relancer** (ou envelopper, section 3.9) quand une couche doit ajouter du contexte.

```java
// Fichier : ParsePercent.java
/** Valide une remise saisie : soit un pourcentage correct, soit une exception explicite. */
static int parsePercent(String input) {
    final int percent;
    try {
        percent = Integer.parseInt(input.strip());
    } catch (NumberFormatException e) {
        throw new IllegalArgumentException("Pourcentage non numérique", e); // on garde la cause
    }
    if (percent < 0 || percent > 100) {
        throw new IllegalArgumentException("Pourcentage hors bornes : " + percent);
    }
    return percent;
}

void main() {
    IO.println(parsePercent(" 15 ")); // 15
    for (String input : new String[] {"dix", "150"}) {
        try {
            parsePercent(input);
        } catch (IllegalArgumentException e) {
            IO.println("Refusé : " + e.getMessage());
        }
    }
    // Refusé : Pourcentage non numérique
    // Refusé : Pourcentage hors bornes : 150
}
```

> 🔒 **Sécurité** — Un `catch` vide est aussi une **faille** : une tentative d'accès refusée, une signature invalide, un paiement rejeté qui « disparaissent » ne laissent aucune trace exploitable par l'équipe de sécurité. OWASP classe l'absence de journalisation des échecs dans son Top 10 (*Security Logging and Monitoring Failures*). Le SEI CERT en fait une règle : **ERR00-J, ne pas supprimer ni ignorer les exceptions vérifiées**.

3.9 🔵 Envelopper une exception technique sans perdre la cause

Une couche technique (fichier, base de données) lève des exceptions techniques (`IOException`, `SQLException`). La couche métier au-dessus n'a pas à connaître ces détails : on **enveloppe** l'exception dans une exception de plus haut niveau, en passant l'originale comme **cause**.

```java
// Fichier : WrapCause.java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;

/** Panne technique : le menu ne peut pas être chargé. */
class MenuUnavailableException extends RuntimeException {
    MenuUnavailableException(String message, Throwable cause) {
        super(message, cause);
    }
}

String loadMenu(Path file) {
    try {
        return Files.readString(file);
    } catch (IOException e) {
        throw new MenuUnavailableException("Menu indisponible", e); // la cause
est conservée
    }
}

void main() {
    try {
        loadMenu(Path.of("menu-qui-n-existe-pas.csv"));
    } catch (MenuUnavailableException e) {
        IO.println(e.getMessage());                              // Menu
indisponible
        IO.println(e.getCause().getClass().getSimpleName());    //
NoSuchFileException
    }
}
```

Deux pièges :

- ❌ `throw new MenuUnavailableException("Menu indisponible", null)` ou sans cause : la pile d'appels d'origine est perdue, le diagnostic devient impossible.
- ❌ **Changer le sens de l'erreur.** Une base de données en panne n'est **pas** une commande introuvable. Transformer une `SQLException` en `OrderNotFoundException` ferait croire à l'utilisateur (et aux logs) que la commande n'existe pas, alors que c'est le serveur qui est malade. Absence → `Optional` ou `OrderNotFoundException` ; panne → une exception technique comme `MenuUnavailableException` ou `UncheckedIOException`.

3.10 🟣 Exceptions checked dans les lambdas

Les interfaces de `java.util.function` ne déclarent **aucune** exception checked. Une lambda qui appelle une méthode levant une `IOException` ne compile donc pas :

```
// ❌ Ne compile pas : unreported exception IOException
List<String> contents = files.stream()
        .map(f -> Files.readString(f))
        .toList();
```

Trois solutions, de la plus simple à la plus générale :

```java
// Fichier : CheckedInLambda.java
import java.io.IOException;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.ArrayList;
import java.util.List;

// Solution 2 : une méthode qui enveloppe dans UncheckedIOException (prévue
pour ça par le JDK)
static String readOrFail(Path file) {
    try {
        return Files.readString(file);
    } catch (IOException e) {
        throw new UncheckedIOException(e);
    }
}

void main() throws IOException {
    Path a = Files.createTempFile("menu-a", ".txt");
    Path b = Files.createTempFile("menu-b", ".txt");
    try {
        Files.writeString(a, "Margherita");
        Files.writeString(b, "Burger");
        List<Path> files = List.of(a, b);

        // Solution 1 : une simple boucle, l'IOException est déclarée par main
        List<String> viaLoop = new ArrayList<>();
        for (Path f : files) {
            viaLoop.add(Files.readString(f));
        }

        // Solution 2 : la lambda appelle la méthode qui enveloppe
        List<String> viaWrapper = files.stream().map(f ->
readOrFail(f)).toList();

        IO.println(viaLoop + " " + viaWrapper); // [Margherita, Burger]
[Margherita, Burger]
    } finally {
        Files.deleteIfExists(a);
        Files.deleteIfExists(b);
    }
}
```

La solution 3, écrire sa propre interface fonctionnelle `ThrowingFunction<T, R, E extends Exception>`, est élégante mais oblige à écrire des adaptateurs : réservez-la aux bibliothèques.

> Ne contournez **jamais** le problème avec un `catch (IOException e) { return null; }` dans la lambda : c'est la valeur magique de la section 3.8, cachée dans un stream.

3.11 🟣 La variable anonyme `_` (Java 22)

Depuis Java 22 (JEP 456, finale), le caractère `_` peut remplacer le nom d'une variable **qu'on n'utilise pas**. Il dit au lecteur (et au compilateur) : « c'est voulu ».

```
// Fichier : UnnamedVariable.java
import java.util.LinkedHashMap;
import java.util.Map;

/** Ici, l'exception EST la réponse : on n'a pas besoin de son contenu. */
static boolean isValidQuantity(String input) {
    try {
        return Integer.parseInt(input) > 0;
    } catch (NumberFormatException _) {
        return false;
    }
}

void main() {
    IO.println(isValidQuantity("3") + " " + isValidQuantity("trois")); // true
false

    Map<String, Integer> cart = new LinkedHashMap<>();
    cart.put("Margherita", 2);
    cart.put("Tiramisu", 1);

    // Paramètre de lambda inutilisé : on ne veut que les quantités
    int[] total = {0};
    cart.forEach((_, qty) -> total[0] += qty);
    IO.println(total[0]); // 3

    // Variable de boucle inutilisée : on compte les lignes
    int lines = 0;
    for (var _ : cart.entrySet()) {
        lines++;
    }
    IO.println(lines); // 2
}
```

> **Attention** : `catch (XxxException _)` ne doit pas devenir une façon élégante d'**avaler** une exception (section 3.8). Il est légitime seulement quand l'exception est une réponse

> attendue et entièrement traitée (ici : « non, ce n'est pas un nombre »). Et l'astuce du tableau `int[] total` sert uniquement à illustrer `_` : pour sommer, préférez `cart.values().stream().mapToInt(Integer::intValue).sum()`.

---

4. Journaliser sans fuir

4.1 🟢 `System.Logger` plutôt que `printStackTrace`

Le problème. `e.printStackTrace()` et `System.out.println` écrivent sur la console, sans date, sans niveau de gravité, sans possibilité de filtrer ni de rediriger. En production, cette sortie est perdue ou mélangée à tout le reste. Et, pire, une pile d'appels affichée **à l'utilisateur** révèle le fonctionnement interne de l'application (section 4.2).

La solution : un logger. Un *logger* écrit des messages **datés**, avec un **niveau** et le **nom de la classe** d'origine, vers une destination configurable (console, fichier, outil de supervision). Le JDK en fournit un, sans dépendance : `System.Logger`.

Niveau	Usage
`ERROR`	une opération a échoué et nécessite une intervention
`WARNING`	anomalie traitée, mais à surveiller
`INFO`	événement métier normal et important (commande confirmée)
`DEBUG`, `TRACE`	détails pour le développement, désactivés en production

```java
// Fichier : LoggerDemo.java (avec SampleData.java, DeliveryPlatform.java,
Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java,
User.java, Dish.java, DishSize.java)
import java.lang.System.Logger;
import java.lang.System.Logger.Level;

class DeliveryService {
    // Un logger par classe, nommé d'après la classe
    private static final Logger LOG =
System.getLogger(DeliveryService.class.getName());

    private final DeliveryPlatform platform;

    DeliveryService(DeliveryPlatform platform) { this.platform = platform; }

    /** Le livreur signale une livraison. */
    void markDelivered(String orderId) {
        Order order = platform.getOrderById(orderId);       // peut lever
OrderNotFoundException
        try {
            order.moveTo(OrderStatus.DELIVERED);
            LOG.log(Level.INFO, "Commande {0} livrée", order.getId());
        } catch (IllegalStateException e) {
            // On journalise AVEC l'exception (3e argument) : la pile part dans
les logs
            LOG.log(Level.WARNING, "Livraison refusée pour la commande " +
order.getId(), e);
            throw e; // et on relance : l'appelant doit savoir que ça a échoué
        }
    }
}

void main() {
    DeliveryPlatform platform = SampleData.platform();
    var service = new DeliveryService(platform);

    String inDelivery =
platform.findOrdersByStatus(OrderStatus.IN_DELIVERY).getFirst().getId();
    String cancelled =
platform.findOrdersByStatus(OrderStatus.CANCELLED).getFirst().getId();

    service.markDelivered(inDelivery);    // INFO dans les logs
    try {
        service.markDelivered(cancelled); // WARNING + pile dans les logs, puis
exception
    } catch (IllegalStateException e) {
        IO.println("Réponse à l'utilisateur : livraison impossible pour cette
commande.");
```

```
    }
}
```

Par défaut, `System.Logger` s'appuie sur le module `java.logging` et écrit sur la sortie d'erreur, par exemple :

```
oct. 24, 2026 9:30:12 PM DeliveryService markDelivered
INFO: Commande order-3f1c… livrée
oct. 24, 2026 9:30:12 PM DeliveryService markDelivered
WARNING: Livraison refusée pour la commande order-8a2b…
java.lang.IllegalStateException: Transition interdite : CANCELLED -> DELIVERED
        at Order.moveTo(Order.java:…)
        …
```

Les deux formes de `log` à connaître :

- `LOG.log(Level.INFO, "Commande {0} livrée", id)` : message avec paramètres `{0}`, `{1}`… (format de `java.text.MessageFormat`).
- `LOG.log(Level.WARNING, "message", exception)` : message + exception, pour garder la pile d'appels **dans les logs**.

Pièges :

- ❌ `LOG.log(Level.WARNING, "Erreur : " + e.getMessage())` : on perd la pile et le type de l'exception. ✅ passer `e` en dernier argument.
- ❌ **Journaliser puis relancer à chaque couche** : la même erreur apparaît cinq fois dans les logs. Journalisez **une fois**, là où l'erreur est traitée (ou, comme ici, là où l'on ajoute un contexte utile).
- 🔵 Avec le format `{0}`, l'apostrophe est un caractère spécial de `MessageFormat` : `"Livraison de l'utilisateur {0}"` affiche « Livraison de lutilisateur {0} ». Doublez-la (`''`) ou évitez-la dans les messages paramétrés.

🔵 **Et SLF4J, Logback, Log4j ?** En entreprise, vous croiserez surtout SLF4J (une façade) avec Logback ou Log4j 2 (des implémentations), qui offrent la syntaxe `log.info("Commande {} livrée", id)`. Le principe est identique. `System.Logger` est lui aussi une façade : il peut être branché sur ces bibliothèques sans changer le code. Dans ce cours, on reste sur le JDK, sans dépendance.

> 🔒 **Sécurité** — Log4Shell (CVE-2021-44228, décembre 2021) : une version de Log4j 2 interprétait des expressions `${jndi:...}` contenues **dans les messages journalisés**. Un attaquant qui faisait journaliser une chaîne de son choix (un en-tête HTTP, un pseudo)

> obtenait l'exécution de code à distance. Leçon : une donnée utilisateur journalisée est une **entrée non fiable**, et une bibliothèque de logs se met à jour comme le reste.

4.2 🔒 Fuite d'information par les messages d'erreur (CWE-209)

Le problème. Le message d'une exception, et surtout sa pile d'appels, sont écrits **pour le développeur**. Ils contiennent des noms de classes, des chemins de fichiers, des versions de bibliothèques, des requêtes SQL, des noms de serveurs. Renvoyés à l'utilisateur, ils deviennent une **carte du système** pour un attaquant. C'est la faiblesse CWE-209 : *Generation of Error Message Containing Sensitive Information*.

Autre fuite classique : des messages **différents** selon la cause. « Commande inexistante » contre « Cette commande appartient à un autre client » permet à un attaquant de deviner quels identifiants existent (énumération).

La solution : deux messages, deux destinataires.

	Destinataire	**Contenu**
Log (serveur)	l'équipe technique	tout le détail : message, exception, pile, identifiant d'incident
Réponse (utilisateur)	le client	un message neutre, utile, et une **référence** d'incident

La référence d'incident (un `UUID`) fait le lien : le client la communique au support, qui retrouve le détail dans les logs.

```java
// Fichier : ErrorResponses.java (avec SampleData.java, DeliveryPlatform.java, Order.java, OrderStatus.java, OrderNotFoundException.java, Customer.java, User.java, Dish.java, DishSize.java)
import java.lang.System.Logger;
import java.lang.System.Logger.Level;
import java.util.UUID;
import java.util.function.Function;

/** Frontière de l'application : transforme une requête en réponse texte (façon HTTP). */
class OrderEndpoint {
    private static final Logger LOG =
System.getLogger(OrderEndpoint.class.getName());
    private final Function<String, Order> lookup;

    OrderEndpoint(Function<String, Order> lookup) { this.lookup = lookup; }

    // ❌ Le message technique part tel quel chez l'utilisateur
    String statusLeaky(String orderId) {
        try {
            return "200 " + lookup.apply(orderId).getStatus();
        } catch (RuntimeException e) {
            return "500 " + e;   // type, message interne... (et parfois toute la pile)
        }
    }

    // ✅ Détail dans les logs, message neutre et référence pour l'utilisateur
    String status(String orderId) {
        try {
            return "200 " + lookup.apply(orderId).getStatus();
        } catch (OrderNotFoundException e) {
            return "404 Commande introuvable.";
        } catch (RuntimeException e) {
            String incident = UUID.randomUUID().toString();
            LOG.log(Level.ERROR, "Incident " + incident + " sur la lecture d'une commande", e);
            return "500 Une erreur est survenue. Référence : " + incident;
        }
    }
}

void main() {
    // Une recherche qui tombe en panne : la base de données ne répond pas
    Function<String, Order> brokenDatabase = id -> {
        throw new IllegalStateException(
                "Connexion refusée : jdbc:postgresql://db-prod-02:5432/foodfast (user=ff_admin)");
    };
```

```
    var endpoint = new OrderEndpoint(brokenDatabase);

    IO.println(endpoint.statusLeaky("order-42"));
    // 500 java.lang.IllegalStateException: Connexion refusée :
jdbc:postgresql://db-prod-02:5432/...

    IO.println(endpoint.status("order-42"));
    // 500 Une erreur est survenue. Référence : 6f0c2e9a-...

    var normal = new OrderEndpoint(SampleData.platform()::getOrderById);
    IO.println(normal.status("order-inconnue")); // 404 Commande introuvable.
}
```

La version ❌ vient de révéler à n'importe qui le type de base, le nom du serveur, son port, le nom de la base et le compte utilisé. La version ✅ ne renvoie rien d'exploitable, et l'équipe technique a tout dans les logs.

Remarquez aussi que la réponse 404 ne **renvoie pas** l'identifiant saisi par l'utilisateur : réafficher une entrée non fiable dans une page web est la porte ouverte à l'injection de script (XSS).

> 🔒 **Sécurité** — Règle : **jamais** de `e.getMessage()`, de `e.toString()` ni de pile d'appels dans une réponse destinée à l'utilisateur. Les frameworks web ont des réglages pour cela (par exemple, ne pas inclure la pile dans les pages d'erreur), vérifiez-les en production. Voir le SEI CERT **ERR01-J** (*Do not allow exceptions to expose sensitive information*).

4.3 🔒 Pas de données personnelles dans les logs

Le problème. Les logs sont copiés, archivés, envoyés vers des outils tiers, lus par de nombreuses personnes (développeurs, exploitation, prestataires). Ils sont rarement protégés comme une base de données. Une adresse e-mail, une adresse postale, un numéro de téléphone ou, pire, un mot de passe ou un numéro de carte qui y apparaissent se retrouvent exposés, et l'entreprise viole le RGPD (minimisation des données).

```
// ❌ Données personnelles en clair dans les logs
LOG.log(Level.INFO, "Livraison pour " + c.getName() + " <" + c.getEmail() + ">,
"
        + c.getAddress());

// ✅ Un identifiant technique suffit pour enquêter ; le détail est en base,
protégé
LOG.log(Level.INFO, "Livraison de la commande {0} pour le client {1}",
        order.getId(), c.getId());
```

Même vigilance avec les `toString()` : un `record` génère un `toString()` qui affiche **tous** ses champs. Un `record Customer(String name, String email, String address)` journalisé « par commodité » fuirait tout. Si une classe contient des données sensibles, redéfinissez `toString()` pour les masquer (séance 2).

> 🔒 **Sécurité** — Ne jamais journaliser : mots de passe (même erronés : c'est souvent le vrai mot de passe avec une faute de frappe), jetons de session ou d'API, numéros de carte, données de santé. En cas de doute, journalisez un **identifiant**, pas la donnée.

4.4 🔒 Injection dans les logs (CWE-117)

Le problème. Un log est un fichier texte, une ligne par événement. Si une **entrée utilisateur** contenant des retours à la ligne est journalisée telle quelle, l'attaquant peut **fabriquer de fausses lignes** de log : masquer son activité, accuser quelqu'un d'autre, ou tromper un outil d'analyse automatique. C'est la faiblesse CWE-117 : *Improper Output Neutralization for Logs*.

```java
// Fichier : LogInjection.java
import java.lang.System.Logger;
import java.lang.System.Logger.Level;

static final Logger LOG = System.getLogger("OrderLookup");

/** Neutralise les caractères de contrôle (\n, \r, tabulations...) d'une entrée non fiable. */
static String forLog(String untrusted) {
    String clean = untrusted.replaceAll("\\p{Cntrl}", "_");
    return clean.length() > 100 ? clean.substring(0, 100) + "…" : clean;
}

void main() {
    // L'identifiant vient d'une URL : l'attaquant y glisse un retour à la ligne
    String orderId = "order-42\nINFO: Remboursement de 500 EUR valide pour order-42";

    // ❌ Le log contient une fausse ligne "INFO: Remboursement..." indiscernable d'une vraie
    LOG.log(Level.WARNING, "Commande introuvable : " + orderId);

    // ✅ L'entrée est neutralisée : tout reste sur une seule ligne
    LOG.log(Level.WARNING, "Commande introuvable : " + forLog(orderId));
}
```

Sortie (sur la sortie d'erreur) :

```
… OrderLookup main
WARNING: Commande introuvable : order-42
INFO: Remboursement de 500 EUR valide pour order-42          <- fausse ligne !
… OrderLookup main
WARNING: Commande introuvable : order-42_INFO: Remboursement de 500 EUR valide pour order-42
```

Les défenses :

1. **Neutraliser** les caractères de contrôle de toute donnée non fiable avant de la journaliser, et **limiter sa longueur** (sinon un attaquant remplit le disque avec des entrées de plusieurs mégaoctets).
2. Mieux : **valider** l'entrée à la frontière. Un identifiant de commande a un format connu (`order-` suivi d'un UUID) ; ce qui ne le respecte pas est refusé avant même d'être journalisé.

3. En production, les logs **structurés** (JSON, un champ par donnée) échappent les caractères spéciaux par construction.

> 🔒 **Sécurité** — Ce problème rejoint `OrderNotFoundException` : son message contient l'identifiant recherché, qui vient souvent de l'utilisateur. Si vous journalisez `e.getMessage()`, validez ou neutralisez l'identifiant **avant** de construire l'exception, ou à la frontière.

---

5. Lire et écrire des fichiers

Jusqu'ici, tout disparaît quand le programme s'arrête : les commandes de `DeliveryPlatform` vivent en mémoire. Pour les garder, il faut les écrire sur le disque, puis les relire au démarrage. Java le fait avec deux classes de `java.nio.file` : **`Path`** désigne un fichier, **`Files`** agit dessus.

5.1 🟢 `Path` : désigner un fichier

Le problème. Un chemin construit en collant des chaînes (`"data" + "/" + name`) se trompe de séparateur sous Windows, oublie un `/`, ou en met deux.

La solution. Un `Path` est un chemin, découpé en éléments. Il se construit avec `Path.of(...)` et se prolonge avec `resolve(...)`. Créer un `Path` ne touche **pas** au disque : le fichier peut ne pas exister.

```java
// Fichier : PathDemo.java
import java.nio.file.Path;

void main() {
    Path dir = Path.of("data");
    Path menu = dir.resolve("menu.csv");              // data/menu.csv

    IO.println(menu);                                 // data/menu.csv
    IO.println(menu.getFileName());                   // menu.csv
    IO.println(menu.getParent());                     // data
    IO.println(menu.isAbsolute());                    // false : relatif au
dossier courant

    // normalize() retire les "." et les ".." inutiles
    Path messy = Path.of("data", "..", "data", ".", "menu.csv");
    IO.println(messy);                                //
data/../data/./menu.csv
    IO.println(messy.normalize());                    // data/menu.csv
}
```

Un chemin **relatif** (`data/menu.csv`) part du dossier courant : celui d'où le programme est lancé. Dans IntelliJ et avec Maven, c'est la racine du projet. `toAbsolutePath()` donne le chemin complet.

5.2 🟢 `Files` : lire et écrire du texte

Pour un petit fichier texte, une ligne suffit. Toutes ces méthodes utilisent **UTF-8** par défaut, et lèvent une `IOException` (checked, §3.2) si le disque refuse.

Méthode	Effet
`Files.writeString(file, texte)`	crée ou **remplace** le fichier
`Files.readString(file)`	tout le contenu, en une `String`
`Files.readAllLines(file)`	une `List<String>`, une ligne par élément
`Files.lines(file)`	un `Stream<String>` à fermer (§3.4)
`Files.exists(file)`	le fichier existe-t-il ?
`Files.createDirectories(dir)`	crée le dossier et ses parents, sans erreur s'il existe déjà
`Files.deleteIfExists(file)`	supprime, sans erreur s'il n'existe pas

Exemple FoodFast : sauvegarder le menu, puis le relire.

```
// Fichier : MenuText.java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.NoSuchFileException;
import java.nio.file.Path;
import java.util.List;

void main() throws IOException {
    Path dir = Files.createTempDirectory("foodfast");      // un dossier
jetable pour l'exemple
    Path menu = dir.resolve("data").resolve("menu.csv");

    Files.createDirectories(menu.getParent());             // sinon :
NoSuchFileException
    Files.writeString(menu, "Margherita;12.50\nCrème brûlée;6.00\n");

    List<String> lines = Files.readAllLines(menu);
    IO.println(lines.size() + " plats : " + lines);        // 2 plats :
[Margherita;12.50, Crème brûlée;6.00]

    // ❌ Lire un fichier absent : une exception, pas une liste vide
    try {
        Files.readString(dir.resolve("absent.csv"));
    } catch (NoSuchFileException e) {
        IO.println("Introuvable : " + e.getFile().endsWith("absent.csv"));   //
Introuvable : true
    }

    // ✅ Quand l'absence est normale (premier démarrage), on la teste avant
    Path orders = dir.resolve("orders.csv");
    List<String> saved = Files.exists(orders) ? Files.readAllLines(orders) :
List.of();
    IO.println(saved.size() + " commande(s) rechargée(s)");     // 0
commande(s) rechargée(s)
}
```

5.3 🟢 Les flux d'octets : `newOutputStream` et `newInputStream`

`writeString` écrit tout d'un coup. Quand on veut écrire **au fur et à mesure**, ou réutiliser une classe qui écrit dans un `OutputStream` (un `RespWriter`, par exemple), on ouvre un **flux** sur le fichier :

- `Files.newOutputStream(file)` : un `OutputStream`, qui crée ou remplace le fichier ;
- `Files.newInputStream(file)` : un `InputStream`, pour le relire.

Un flux est une ressource : il se ferme avec **try-with-resources** (§3.4). Et c'est la fermeture qui garantit que tout est bien écrit.

```
// Fichier : MenuStream.java
import java.io.BufferedOutputStream;
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.nio.charset.StandardCharsets;
import java.nio.file.Files;
import java.nio.file.Path;

static void writeMenu(OutputStream out) throws IOException {
    for (String dish : new String[] {"Margherita", "Burger", "Tiramisu"}) {
        out.write((dish + "\n").getBytes(StandardCharsets.UTF_8));   //
toujours un charset explicite
    }
}

void main() throws IOException {
    Path dir = Files.createTempDirectory("foodfast");

    // ❌ Flux jamais fermé : les octets restent dans le tampon, le fichier est
vide
    Path lost = dir.resolve("lost.txt");
    OutputStream forgotten = new
BufferedOutputStream(Files.newOutputStream(lost));
    writeMenu(forgotten);
    IO.println("Sans close : " + Files.size(lost) + " octets");      // Sans
close : 0 octets

    // ✅ try-with-resources : close() vide le tampon, même en cas d'exception
    Path menu = dir.resolve("menu.txt");
    try (OutputStream out = new
BufferedOutputStream(Files.newOutputStream(menu))) {
        writeMenu(out);
    }
    IO.println("Avec close : " + Files.size(menu) + " octets");      // Avec
close : 27 octets

    try (InputStream in = Files.newInputStream(menu)) {
        String content = new String(in.readAllBytes(), StandardCharsets.UTF_8);
        IO.println(content.lines().toList());                        //
[Margherita, Burger, Tiramisu]
    }
}
```

> 💡 Le `BufferedOutputStream` regroupe les petites écritures en une seule écriture disque : beaucoup plus rapide quand on écrit ligne par ligne. Le prix : rien n'est écrit tant que le tampon n'est pas vidé, par `flush()` ou `close()`.

5.4 🔵 Écrire de façon atomique

Le problème. `Files.writeString(menu, ...)` commence par **vider** le fichier, puis écrit. Si le programme est tué entre les deux (coupure de courant, `kill`, disque plein), il reste un fichier vide ou à moitié écrit. L'ancienne sauvegarde est perdue, et la nouvelle est illisible.

La solution. Écrire dans un fichier **temporaire**, puis le **renommer** sur le fichier final. Avec l'option `ATOMIC_MOVE`, le renommage se fait d'un seul coup : à tout instant, le fichier final est soit l'ancienne version complète, soit la nouvelle, jamais un mélange.

```java
// Fichier : AtomicSave.java
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

/** Remplace le contenu de file sans jamais laisser un fichier à moitié écrit.
*/
static void saveAtomically(Path file, String content) throws IOException {
    // Le temporaire est créé dans le MÊME dossier : un renommage atomique ne
traverse pas les disques
    Path tmp = Files.createTempFile(file.getParent(), "save-", ".tmp");
    try {
        Files.writeString(tmp, content);
        Files.move(tmp, file, StandardCopyOption.ATOMIC_MOVE,
StandardCopyOption.REPLACE_EXISTING);
    } finally {
        Files.deleteIfExists(tmp);   // si l'écriture a échoué, on ne laisse
pas de déchet
    }
}

void main() throws IOException {
    Path menu = Files.createTempDirectory("foodfast").resolve("menu.csv");
    saveAtomically(menu, "Margherita;12.50\n");
    saveAtomically(menu, "Margherita;12.50\nBurger;11.90\n");
    IO.println(Files.readAllLines(menu));                  //
[Margherita;12.50, Burger;11.90]
    try (var files = Files.list(menu.getParent())) {
        IO.println(files.count() + " fichier dans le dossier");   // 1 fichier
dans le dossier
    }
}
```

Après un renommage réussi, `tmp` n'existe plus : le `deleteIfExists` du `finally` ne fait alors rien. Les bases de données et Redis lui-même sauvegardent ainsi.

5.5 🔒 Un nom de fichier choisi par l'utilisateur (CWE-22)

Le problème. FoodFast permet de télécharger une facture par son nom : `invoices.resolve(name)`. Un attaquant envoie `../../etc/passwd`. `resolve` ne vérifie rien : le chemin **sort** du dossier des factures et désigne n'importe quel fichier du serveur. C'est la **traversée de répertoire** (CWE-22), l'une des failles web les plus courantes. En écriture, c'est pire : l'attaquant choisit où écrire.

La solution. Normaliser le chemin obtenu, puis vérifier qu'il est **toujours dans** le dossier autorisé.

```
// Fichier : SafeInvoice.java
import java.nio.file.Path;

/** Le fichier name, à condition qu'il soit dans dir. Sinon :
IllegalArgumentException. */
static Path resolveInside(Path dir, String name) {
    Path base = dir.toAbsolutePath().normalize();
    Path target = base.resolve(name).normalize();
    if (!target.startsWith(base) || target.equals(base)) {
        throw new IllegalArgumentException("chemin refusé : " + name);
    }
    return target;
}

void main() {
    Path invoices = Path.of("invoices");
    for (String name : new String[] {"2026-0042.pdf", "../../etc/passwd",
"/etc/passwd", "."}) {
        try {
            IO.println("✅ " + resolveInside(invoices, name).getFileName());
        } catch (IllegalArgumentException e) {
            IO.println("❌ " + e.getMessage());
        }
    }
}
```

Sortie :

```
✅ 2026-0042.pdf
❌ chemin refusé : ../../etc/passwd
❌ chemin refusé : /etc/passwd
❌ chemin refusé : .
```

Trois détails comptent :

- **`normalize()` avant la vérification** : sinon `invoices/../../etc/passwd` « commence » bien par `invoices`.
- **`Path.startsWith` compare des éléments**, pas des caractères : `invoices2/x.pdf` ne commence pas par `invoices`. Avec `String.startsWith`, il passerait.
- Un nom **absolu** (`/etc/passwd`) remplace complètement la base dans `resolve` : la vérification l'attrape aussi.

> 🔒 **Encore mieux** : quand c'est possible, ne pas accepter de chemin du tout. Une liste blanche (« un nom de fichier simple : lettres, chiffres, `-`, `.` ») ou un identifiant traduit côté serveur ferment la porte avant même de construire le chemin.

5.6 🟢 Tester avec `@TempDir`

Un test qui écrit dans `data/` laisse des fichiers derrière lui, et échoue si un autre test a écrit au même endroit. JUnit fournit un dossier **neuf et vide pour chaque test**, supprimé ensuite : un paramètre (ou un attribut) annoté `@TempDir`.

La classe testée : un petit stockage du menu, qui rend une liste vide au premier démarrage.

```
// Fichier : MenuFile.java (compilation seule)
import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;

class MenuFile {
    private final Path file;

    MenuFile(Path file) {
        this.file = file;
    }

    void save(List<String> dishes) throws IOException {
        Files.createDirectories(file.getParent());
        Files.write(file, dishes);                // une ligne par plat, en
UTF-8
    }

    /** Le menu enregistré, ou une liste vide s'il n'y a encore rien. */
    List<String> load() throws IOException {
        return Files.exists(file) ? Files.readAllLines(file) : List.of();
    }
}
```

```java
// Fichier : MenuFileTest.java (avec MenuFile.java)
import static org.junit.jupiter.api.Assertions.assertEquals;

import java.io.IOException;
import java.nio.file.Path;
import java.util.List;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.io.TempDir;

class MenuFileTest {

    @TempDir
    Path dir;   // un dossier vide, créé avant chaque test et supprimé après

    @Test
    void le_menu_relu_est_celui_qui_a_ete_sauvegarde() throws IOException {
        MenuFile menu = new MenuFile(dir.resolve("data/menu.txt"));
        menu.save(List.of("Margherita", "Crème brûlée"));
        assertEquals(List.of("Margherita", "Crème brûlée"), menu.load());
    }

    @Test
    void au_premier_demarrage_le_menu_est_vide() throws IOException {
        assertEquals(List.of(), new MenuFile(dir.resolve("menu.txt")).load());
    }

    @Test
    void un_nouveau_save_remplace_l_ancien() throws IOException {
        MenuFile menu = new MenuFile(dir.resolve("menu.txt"));
        menu.save(List.of("Margherita", "Burger"));
        menu.save(List.of("Tiramisu"));
        assertEquals(List.of("Tiramisu"), menu.load());
    }
}
```

Chaque test reçoit son propre dossier : ils peuvent s'exécuter dans n'importe quel ordre sans se gêner. C'est exactement ce qu'il faut pour tester la sauvegarde du TP.

---

6. Ce qu'il faut retenir

Lambdas

- Une **interface fonctionnelle** a une seule méthode abstraite (`Discount`, `Predicate`, `Function`, `Comparator`…). Une **lambda** en est une implémentation concise : `t ->`

`t.subtract(...)`.

- Une lambda lit des variables **effectivement finales**, elle ne les modifie pas.
- Une **référence de méthode** (`Order::getTotal`, `Integer::sum`, `BigDecimal::add`) remplace une lambda qui ne fait qu'appeler une méthode.

Streams

- Source → opérations intermédiaires (**paresseuses**) → **une** opération terminale. Un stream ne modifie pas sa source et ne sert qu'une fois.
- `filter`, `map`, `sorted(Comparator.comparing(...))`, `limit`, `toList()` (non modifiable), `count`, `anyMatch`.
- Somme de montants : `.map(Order::getTotal).reduce(BigDecimal.ZERO, BigDecimal::add)`.
- Regroupement : `collect(groupingBy(Order::getStatus, counting()))`. Ordre non garanti.
- Pas d'effets de bord dans les lambdas ; `parallelStream()` seulement après mesure.

Exceptions

- Checked (`IOException`) : le monde extérieur, traitement obligatoire. Unchecked (`RuntimeException`) : bug, contrat ou règle métier violés.
- On attrape là où l'on **sait quoi faire**, du plus spécifique au plus général. Les ressources se ferment avec **try-with-resources**.
- `OrderNotFoundException` (unchecked) quand la commande **doit** exister ; `Optional` quand l'absence est **normale**.
- `IllegalArgumentException` : mauvais paramètre. `IllegalStateException` : mauvais moment (transition interdite).
- **Jamais** de `catch` vide, de `return 0` ou `return null` « par défaut », de `printStackTrace`.

Fichiers

- `Path.of(...)` et `resolve(...)` désignent un fichier ; `Files` lit et écrit (UTF-8 par défaut, `IOException` checked).
- Un flux (`newOutputStream`, `newInputStream`) se ferme avec try-with-resources : c'est `close()` qui garantit l'écriture.
- Sauvegarde sûre : fichier temporaire dans le même dossier, puis `Files.move(..., ATOMIC_MOVE, REPLACE_EXISTING)`.
- 🔒 Nom de fichier venu de l'extérieur : `normalize()`, puis `startsWith(dossier)` (CWE-22).

- Tests : `@TempDir`, un dossier neuf par test.

Journalisation et sécurité

- `System.getLogger(...)`, un niveau, et l'exception en dernier argument.
- 🔒 Détail technique dans les logs, message neutre + référence pour l'utilisateur (CWE-209).
- 🔒 Pas de données personnelles dans les logs ; un identifiant suffit.
- 🔒 Neutraliser les retours à la ligne des entrées utilisateur avant de les journaliser (CWE-117).

---

7. Pour aller plus loin

- **dev.java** : Lambda Expressions, The Stream API, Exceptions, Java I/O.
- **Documentation Java 25** : `java.util.stream`, `Collectors`, `System.Logger`.
- **JEP** : JEP 485 — Stream Gatherers, JEP 456 — Unnamed Variables & Patterns, JEP 458 — Launch Multi-File Source-Code Programs.
- **OWASP** : Error Handling Cheat Sheet, Logging Cheat Sheet.
- **SEI CERT Oracle Coding Standard for Java** : chapitre *Exceptional Behavior (ERR)*, en particulier ERR00-J, ERR01-J et ERR08-J (ne pas attraper `NullPointerException`).
- **MITRE CWE** : CWE-209, CWE-117, CWE-390 (erreur détectée sans action).

Défis 🟣

1. Calculez le panier moyen par client (`Map<String, BigDecimal>`) sans jamais passer par `double` : `groupingBy` avec `Collectors.teeing` (somme et nombre), puis une division `BigDecimal` arrondie à 2 décimales.
2. Écrivez un `Gatherer` maison qui ne laisse passer qu'une commande par client (« dédoublonner selon une clé »).
3. Rendez `OrderEndpoint` résistant à l'énumération : la même réponse pour « commande inexistante » et « commande d'un autre client ».

Annexe A — Le modèle FoodFast de cette séance

Ce modèle reprend `cours/modele/FoodFastModel.java` (séance 2), découpé **un type par fichier** pour que le lanceur `java` le retrouve. Les ajouts de la séance 3 sont signalés par `// S3`. `Courier`, `Discount` et ses implémentations ne servent pas dans les exemples ci-dessus et ne sont pas repris.

A.1 `DishSize`

```
// Fichier : DishSize.java (compilation seule)
enum DishSize { SMALL, MEDIUM, LARGE }
```

A.2 `OrderStatus`

```
// Fichier : OrderStatus.java (compilation seule)
enum OrderStatus {
    PENDING, CONFIRMED, PREPARING, IN_DELIVERY, DELIVERED, CANCELLED;

    // S3 : les transitions autorisées du cycle de vie
    boolean canMoveTo(OrderStatus next) {
        return switch (this) {
            case PENDING -> next == CONFIRMED || next == CANCELLED;
            case CONFIRMED -> next == PREPARING || next == CANCELLED;
            case PREPARING -> next == IN_DELIVERY || next == CANCELLED;
            case IN_DELIVERY -> next == DELIVERED;
            case DELIVERED, CANCELLED -> false;
        };
    }
}
```

A.3 `Dish`

```
// Fichier : Dish.java (compilation seule, avec DishSize.java)
import java.math.BigDecimal;
import java.math.RoundingMode;
import java.util.Objects;

/** Valeur immuable : un plat du menu. */
record Dish(String name, BigDecimal price, DishSize size) {
    Dish {
        Objects.requireNonNull(name, "name");
        Objects.requireNonNull(price, "price");
        Objects.requireNonNull(size, "size");
        if (name.isBlank()) throw new IllegalArgumentException("Nom vide");
        if (price.signum() < 0) throw new IllegalArgumentException("Prix
négatif : " + price);
        price = price.setScale(2, RoundingMode.HALF_UP); // 12.5 et 12.50
deviennent égaux
    }
}
```

A.4 `User`

```
// Fichier : User.java (compilation seule)
import java.util.Objects;
import java.util.UUID;

/** Héritage : ce que partagent tous les utilisateurs de la plateforme. */
abstract class User {
    private final UUID id = UUID.randomUUID();
    private final String name;
    private final String email;

    protected User(String name, String email) {
        this.name = Objects.requireNonNull(name);
        this.email = Objects.requireNonNull(email);
    }
    public UUID getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public abstract String role();
}
```

A.5 `Customer`

```java
// Fichier : Customer.java (compilation seule, avec User.java)
import java.util.Objects;

class Customer extends User {
    private final String address;
    Customer(String name, String email, String address) {
        super(name, email);
        this.address = Objects.requireNonNull(address);
    }
    public String getAddress() { return address; }
    @Override public String role() { return "CLIENT"; }
}
```

A.6 `Order`

```
// Fichier : Order.java (compilation seule, avec Customer.java, User.java,
Dish.java, DishSize.java, OrderStatus.java)
import java.math.BigDecimal;
import java.time.LocalDateTime;
import java.util.Collections;
import java.util.LinkedHashMap;
import java.util.Map;
import java.util.Objects;
import java.util.UUID;

/** Entité mutable, encapsulée : une commande. */
class Order {
    private final String id = "order-" + UUID.randomUUID();
    private final Customer customer;
    private final Map<Dish, Integer> items = new LinkedHashMap<>();
    private final LocalDateTime createdAt = LocalDateTime.now();
    private OrderStatus status = OrderStatus.PENDING;

    Order(Customer customer) { this.customer =
Objects.requireNonNull(customer); }

    public void addDish(Dish dish, int quantity) {
        if (status != OrderStatus.PENDING) throw new
IllegalStateException("Commande déjà validée");
        if (quantity <= 0) throw new IllegalArgumentException("Quantité
invalide : " + quantity);
        items.merge(dish, quantity, Integer::sum);
    }

    public BigDecimal getTotal() {
        BigDecimal total = BigDecimal.ZERO;
        for (Map.Entry<Dish, Integer> e : items.entrySet()) {
            total =
total.add(e.getKey().price().multiply(BigDecimal.valueOf(e.getValue())));
        }
        return total;
    }

    public void confirm() {
        if (status != OrderStatus.PENDING || items.isEmpty()) throw new
IllegalStateException("Confirmation impossible");
        status = OrderStatus.CONFIRMED;
    }

    // S3 : changement de statut contrôlé par OrderStatus.canMoveTo
    public void moveTo(OrderStatus next) {
        Objects.requireNonNull(next, "next");
```

```
        if (!status.canMoveTo(next)) {
            throw new IllegalStateException("Transition interdite : " + status
+ " -> " + next);
        }
        if (next == OrderStatus.CONFIRMED && items.isEmpty()) {
            throw new IllegalStateException("Commande vide");
        }
        status = next;
    }

    public String getId() { return id; }
    public Customer getCustomer() { return customer; }
    public Map<Dish, Integer> getItems() { return
Collections.unmodifiableMap(items); }
    public OrderStatus getStatus() { return status; }
    public LocalDateTime getCreatedAt() { return createdAt; }
}
```

A.7 `OrderNotFoundException`

```
// Fichier : OrderNotFoundException.java (compilation seule)
/** S3 : levée quand on exige une commande qui n'existe pas. */
class OrderNotFoundException extends RuntimeException {
    private final String orderId;

    OrderNotFoundException(String orderId) {
        super("Commande introuvable : " + orderId);
        this.orderId = orderId;
    }

    public String getOrderId() { return orderId; }
}
```

A.8 `DeliveryPlatform`

```
// Fichier : DeliveryPlatform.java (compilation seule, avec Order.java,
OrderNotFoundException.java, Customer.java, User.java, Dish.java,
DishSize.java, OrderStatus.java)
import java.util.Collection;
import java.util.Collections;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

class DeliveryPlatform {
    private final Map<String, Order> orders = new HashMap<>();

    public void placeOrder(Order order) { orders.put(order.getId(), order); }

    /** Recherche : l'absence est un résultat normal. */
    public Optional<Order> findOrderById(String id) { return
Optional.ofNullable(orders.get(id)); }

    // S3 : accès, la commande DOIT exister
    public Order getOrderById(String id) {
        return findOrderById(id).orElseThrow(() -> new
OrderNotFoundException(id));
    }

    public Collection<Order> getOrders() { return
Collections.unmodifiableCollection(orders.values()); }

    // S3 : recherches avec l'API Stream
    public List<Order> findOrdersByStatus(OrderStatus status) {
        return orders.values().stream()
                     .filter(o -> o.getStatus() == status)
                     .toList();
    }

    public List<Order> findOrdersByCustomer(Customer customer) {
        return orders.values().stream()
                     .filter(o ->
o.getCustomer().getId().equals(customer.getId()))
                     .toList();
    }
}
```

A.9 `SampleData`

```
// Fichier : SampleData.java (compilation seule, avec DeliveryPlatform.java,
Order.java, OrderNotFoundException.java, Customer.java, User.java, Dish.java,
DishSize.java, OrderStatus.java)
import java.math.BigDecimal;
import java.util.List;

/** Jeu de données de démonstration : 2 clients, 4 plats, 6 commandes. */
final class SampleData {
    static final Dish MARGHERITA = new Dish("Margherita", new
BigDecimal("12.50"), DishSize.MEDIUM);
    static final Dish BURGER = new Dish("Burger", new BigDecimal("11.90"),
DishSize.LARGE);
    static final Dish SALADE = new Dish("Salade César", new BigDecimal("9.80"),
DishSize.SMALL);
    static final Dish TIRAMISU = new Dish("Tiramisu", new BigDecimal("5.50"),
DishSize.SMALL);

    static final Customer ALICE = new Customer("Alice", "alice@example.com",
"10 rue de Paris");
    static final Customer BOB = new Customer("Bob", "bob@example.com", "5
avenue de Lyon");

    private static final List<OrderStatus> HAPPY_PATH = List.of(
            OrderStatus.CONFIRMED, OrderStatus.PREPARING,
OrderStatus.IN_DELIVERY, OrderStatus.DELIVERED);

    private SampleData() { }

    static DeliveryPlatform platform() {
        var platform = new DeliveryPlatform();
        platform.placeOrder(order(ALICE, OrderStatus.DELIVERED, MARGHERITA, 2,
TIRAMISU, 1));
        platform.placeOrder(order(BOB, OrderStatus.DELIVERED, BURGER, 1,
TIRAMISU, 1));
        platform.placeOrder(order(ALICE, OrderStatus.PENDING, SALADE, 1, null,
0));
        platform.placeOrder(order(BOB, OrderStatus.IN_DELIVERY, MARGHERITA, 3,
null, 0));
        platform.placeOrder(order(ALICE, OrderStatus.CANCELLED, BURGER, 1,
TIRAMISU, 2));
        platform.placeOrder(order(BOB, OrderStatus.DELIVERED, BURGER, 2, null,
0));
        return platform;
    }

    /** Crée une commande d'un ou deux plats et la fait avancer jusqu'au statut
voulu. */
```

```
    private static Order order(Customer customer, OrderStatus target,
                               Dish dish1, int qty1, Dish dish2, int qty2) {
        Order order = new Order(customer);
        order.addDish(dish1, qty1);
        if (dish2 != null) order.addDish(dish2, qty2);
        if (target == OrderStatus.CANCELLED) {
            order.moveTo(OrderStatus.CANCELLED);
            return order;
        }
        for (OrderStatus next : HAPPY_PATH) {
            if (order.getStatus() == target) break;
            order.moveTo(next);
        }
        return order;
    }
}
```